A Project Report on

MULTIMODAL DISASTER INTELLIGENCE PLATFORM FOR REAL-TIME DISASTER ASSESSMENT USING ENVIRONMENTAL METADATA, SOCIAL MEDIA, AND SATELLITE IMAGERY

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

MARURI SAI RAMA LINGA REDDY (22BCE7957)

BARMA RAM CHARAN (22BCE8971)

MADA VISHRUTH REDDY (22BCE7584)

VIT-AP UNIVERSITY

School of Computer Science and Engineering

April, 2026

DECLARATION

We hereby declare that the thesis entitled "Multimodal Disaster Intelligence Platform for Real-Time Disaster Assessment Using Environmental Metadata, Social Media, and Satellite Imagery" submitted by MARURI SAI RAMA LINGA REDDY (22BCE7957), BARMA RAM CHARAN (22BCE8971), MADA VISHRUTH REDDY (22BCE7584) for the award of the degree of Bachelor of Technology in Computer Science and Engineering at VIT-AP University is a record of bonafide work carried out under the guidance of S. KALYANI. The work reported in this thesis has not been submitted, either in full or in part, for the award of any other degree or diploma in this institute or any other institution.

Place: Amaravati

Date: _____

Signature(s) of the Candidate(s)

CERTIFICATE

This is to certify that the senior design project titled "**Multimodal Disaster Intelligence Platform for Real-Time Disaster Assessment Using Environmental Metadata, Social Media, and Satellite Imagery**" submitted by **MARURI SAI RAMA LINGA REDDY (22BCE7957), BARMA RAM CHARAN (22BCE8971), MADA VISHRUTH REDDY (22BCE7584)** in partial fulfillment of the requirements for the award of Bachelor of Technology in Computer Science and Engineering at VIT-AP University is a record of bonafide work carried out under my guidance. To the best of my knowledge, the contents of this thesis, in full or in part, have neither been copied from any other source nor submitted to any other institute or university for the award of any degree or diploma.

Dr. YYYY

External Guide

(with company seal)

The thesis is satisfactory / unsatisfactory

Internal Examiner **External Examiner**

Approved by

PROGRAM CHAIR **DEAN**

B. Tech. CSE School Of Computer Science and Engineering

ABSTRACT

What this project does, at its core, is build a disaster assessment system that pulls in three kinds of inputs already sitting in the repository: environmental metadata, social media image-text data, and post-disaster satellite images. These aren't treated as separate tools, that's the key part. The system mashes all of them together into one single result, and that result can be reviewed through a web application we built on top of it. We spent a lot of time getting the integration to feel seamless.

The codebase has three branch models and one fusion block. The environmental branch reads a 32-value input vector drawn from weather, storm, seismic, and hydrological groups. The crisis branch takes one image and one text report as input, the satellite branch handles post-disaster xBD imagery. All three branch outputs get fed into the `TriFusionLayer`, which is where the actual combining happens. It took us a while to get this right because the branches spit out embeddings of different sizes and formats.

What the final system hands back is disaster type, severity, priority, population impact, and resource demand. The numbers we got were honestly pretty strong, based on the saved evaluation files in this repository the full tri-modal setting reaches 99.41% priority accuracy and 0.0398 severity mean absolute error. We also ran ablation experiments. Those showed that the model still runs with missing modalities, which matters a lot for real-world use. And the satellite branch gives the largest gain over the crisis-only setting.

There's also Grad-CAM explanations, a FastAPI server, background analysis jobs, and a browser-based dashboard for running and reviewing cases. So the repository is an integrated prototype for multimodal disaster analysis. Not just a random collection of unrelated models thrown together.

ACKNOWLEDGEMENT

We thank Dr. KALYANI SUNKARA, VIT-AP University, Amaravathi for guiding us throughout this senior design project. The regular discussions on model design, debugging, evaluation, and documentation helped us complete the work in a structured way. My association with him / her is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert.

We also thank the faculty and staff of VIT-AP University, especially the School of Computer Science and Engineering, for providing the environment and resources we needed for this project. Their support helped us bring the research work, software implementation, and documentation together into one complete submission.

We are grateful to our classmates, friends, and family members for their support during training runs, testing, review, and final report preparation.

Place: Amaravati

Date: _____

Name of the students

TABLE OF CONTENTS

DECLARATION 2
CERTIFICATE 3
ABSTRACT 4
ACKNOWLEDGEMENT 5
TABLE OF CONTENTS 6
LIST OF FIGURES 9
LIST OF TABLES 11
LIST OF ACRONYMS 12
CHAPTER 1 13
INTRODUCTION 13
1.1 BACKGROUND AND MOTIVATION 13
1.2 PROBLEM STATEMENT 13
1.3 OBJECTIVES 14
1.4 SCOPE OF THE PROJECT 14
1.5 KEY CONTRIBUTIONS 14
1.6 ORGANIZATION OF THE REPORT 15
CHAPTER 2 16
LITERATURE REVIEW 16
2.1 SOCIAL MEDIA FOR CRISIS UNDERSTANDING 16
2.2 SATELLITE DAMAGE ASSESSMENT 16
2.3 ENVIRONMENTAL AND SENSOR-BASED HAZARD MONITORING 16
2.4 MULTIMODAL FUSION AND CROSS-ATTENTION 17
2.5 EXPLAINABILITY IN HIGH-STAKES AI SYSTEMS 17
2.6 RESEARCH GAP 18
2.7 COMPARATIVE ANALYSIS OF EXISTING DISASTER INTELLIGENCE SYSTEMS 18
CHAPTER 3 21
SYSTEM DESIGN AND DATA PREPARATION 21
3.1 SYSTEM OVERVIEW 21
3.2 PROPOSED SYSTEM ARCHITECTURE 21
3.3 MODALITIES USED IN THE PROJECT 25

3.4 ENVIRONMENTAL DATASET PREPARATION 25
3.5 CRISIS AND SATELLITE DATASETS 26
3.6 DATASET CHARACTERISTICS AND PREPROCESSING DETAILS 26
3.7 FEATURE ENGINEERING STRATEGY 28
3.8 PROBLEM FORMULATION 28
CHAPTER 4 29
PROPOSED METHODOLOGY 29
4.1 ENVIRONMENTAL BRANCH: ADAPTIVEIOTCLASSIFIER 29
4.2 ENVIRONMENTAL FEATURE GROUPING AND CONFIDENCE WEIGHTING 29
4.3 CRISIS BRANCH: ADAPTIVEFUSIONCLASSIFIER 31
4.4 IMAGE-TEXT REPRESENTATION IN THE CRISIS BRANCH 31
4.5 SATELLITE BRANCH: MODIFIED DEEPLABV3+ 32
4.6 TRI-FUSION LAYER 33
4.7 MULTI-TASK PREDICTION HEADS 33
4.8 EXPLAINABILITY AND USER-FACING REASONING 34
4.9 EXPLAINABILITY WORKFLOW 34
CHAPTER 5 37
IMPLEMENTATION AND SYSTEM INTEGRATION 37
5.1 REPOSITORY ORGANIZATION 37
5.2 TRAINING WORKFLOW 37
5.3 INFERENCE PIPELINE 37
5.4 DASHBOARD AND HUMAN INTERACTION LAYER 39
5.5 WEB APPLICATION WORKFLOW 39
5.6 ENGINEERING TRADE-OFFS 45
5.7 BACKEND API AND DEPLOYMENT WORKFLOW 45
CHAPTER 6 48
EXPERIMENTAL RESULTS AND DISCUSSION 48
6.1 EVALUATION STRATEGY 48
6.2 ENVIRONMENTAL BRANCH RESULTS 48
6.3 CRISIS BRANCH RESULTS 50
6.4 SATELLITE BRANCH RESULTS 52
6.5 TRI-FUSION ABLATION RESULTS 53
6.6 ROBUSTNESS AND PRACTICAL OBSERVATIONS 53

6.7 LIMITATIONS 54
6.8 ERROR ANALYSIS AND CASE-LEVEL INTERPRETATION 54
6.9 OPERATIONAL RELEVANCE AND ETHICAL CONSIDERATIONS 56
CHAPTER 7 59
CONCLUSION AND FUTURE WORK 59
7.1 CONCLUSION 59
7.2 FUTURE WORK 59
REFERENCES 60
APPENDICES 63
APPENDIX A REPOSITORY SUMMARY 63
APPENDIX B KEY EXECUTION COMMANDS 63
APPENDIX C MODEL CONFIGURATION SUMMARY 63
APPENDIX D SAMPLE INFERENCE WORKFLOW 64

LIST OF FIGURES

Figure 1.1 Overall architecture of the Multimodal Disaster Intelligence Platform showing the three upstream branches, the fusion layer, and the downstream decision heads. 15

Figure 3.1 Overall architecture of the proposed Multimodal Disaster Intelligence Platform …….. 22

Figure 4.1 Learned sensor-group importance across disaster categories in the environmental branch. The model assigns high weight to the physically relevant feature families for each hazard type. 29

Figure 5.1 Overview page of the web application. It summarizes the latest alert state, detected hazard, response priority, recent incident records, and the recommended navigation flow across the platform. 40

Figure 5.2 New-analysis page, first section. The operator can upload crisis imagery, enter the public report text, and optionally add satellite evidence before running the fused analysis. 41

Figure 5.3 New-analysis page, second section. The form accepts geographic context and environmental measurements so the same workflow can combine crisis, satellite, and environmental inputs in one submission. 42

Figure 5.4 Incident-details page. This screen presents the fused alert level, hazard category, severity, modality contributions, crisis probabilities, sensor weights, and estimated resource demand for the active case. 43

Figure 5.5 Reports page. Saved incident runs can be reviewed later, compared, and exported, which makes the application useful beyond one-time inference. 44

Figure 6.1 Confusion matrix for the environmental metadata classifier. Storm, earthquake, and flood classes are strongly separated, while fire and unknown show the main overlap. 49

Figure 6.2 Per-class ROC curves for the environmental metadata classifier, showing strong class separation across most hazard categories. 49

Figure 6.3 Severity regression analysis for the environmental branch, including agreement between predicted and target severity values and the residual pattern. 49

Figure 6.4 Training behavior of the crisis-media branch, showing stable validation performance and balanced confidence between visual and textual streams. 51

Figure 6.5 Confusion matrix for the crisis-media branch on the humanitarian classification task. 51

Figure 6.6 Training history of the satellite branch, including loss, mean IoU progression, and learning-rate scheduling. 52

LIST OF TABLES

Table 3.1 Complementary role of each modality in the proposed platform 25

Table 3.2 Environmental metadata sources used in the project.....26

Table 4.1 Core configuration of the tri-fusion model 33

Table 5.1 Major implementation components in the repository 38

Table 5.2 Main web routes and their implementation roles.....38

Table 6.1 Crisis branch per-class performance on the humanitarian task 50

Table 6.2 Satellite branch class-wise validation performance.....52

Table 6.3 Ablation study across modality configurations 53

LIST OF ACRONYMS

Acronym	Meaning
API	Application Programming Interface
AUC	Area Under the Receiver Operating Characteristic Curve
BLIP	Bootstrapping Language-Image Pre-training
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CrisisMMD	Crisis Multimodal Twitter Dataset
F1	Harmonic Mean of Precision and Recall
GIS	Geographic Information System
GPU	Graphics Processing Unit
IoT	Internet of Things
MAE	Mean Absolute Error
MHA	Multi-Head Attention
ROC	Receiver Operating Characteristic
ViT	Vision Transformer
XAI	Explainable Artificial Intelligence
xBD	xView2 Building Damage Assessment Dataset
XLM-R	XLM-RoBERTa

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

When a disaster happens, the information you can actually get your hands on is usually scattered everywhere. A social media post might show what people on the ground are seeing. A satellite image captures physical damage over a much wider area. Environmental or seismic readings reflect the hazard condition itself, so they're telling you something completely different from what the images show. The frustrating part is these sources are often reviewed in isolation, even when they're all describing the same event. Nobody is pulling them together in a systematic way.

This project started from that gap. In our repository, the environmental model, the crisis-media model, and the satellite model were each useful on their own. That was clear pretty early on. But none of them gave the full picture by themselves. What we wanted was a single system that could draw on all three modalities and kick out one operational result rather than three separate outputs that someone would have to mentally piece together. After trying a few different approaches to combining them, we settled on the fusion architecture described later in this thesis.

The result is the Multimodal Disaster Intelligence Platform. It stitches together environmental metadata, image-text social reports, and satellite imagery through a tri-modal fusion layer. But we didn't just want better prediction scores. We also wanted the output to be easier for someone to actually review. So the platform includes explanations, saved case history, and a working browser interface that ties everything together. This was one of the trickier parts to get working because the explanation pipeline had to stay in sync with the prediction pipeline.

1.2 PROBLEM STATEMENT

The main problem this project addresses is pretty straightforward. How do you combine disaster evidence from different modalities into one usable assessment? A person using the system shouldn't have to read separate outputs from a sensor model, a social media model, and a satellite model and then try to mentally piece all of that together on their own. That's the exact situation we wanted to fix.

A second problem is missing data. This one turned out to be really important in practice, maybe more than we expected at first. A case may begin with only a social image and a short text caption, satellite data may arrive hours or even days later. Sensor values might be partial or completely unavailable. The system should still hand back a useful result even when some inputs aren't there yet. We spent a lot of time debugging this part to make sure partial inputs didn't crash the pipeline.

1.3 OBJECTIVES

The first goal was to build a single pipeline connecting all three modes of disaster assessment. The idea was to take whatever evidence is available and have the system classify the disaster type, estimate severity, assign a priority level, and suggest resource information. All from one unified model rather than running three separate ones.

The second objective was to make the system usable as a running application. We didn't want it to just be a collection of training notebooks that only work inside Jupyter. So the project includes an API server, background analysis jobs, explanation support, and a five-page web dashboard for running and reviewing cases. Someone could actually sit down and use it end to end. In hindsight, we should have started on the web interface earlier because integrating it at the end caused a bunch of issues.

1.4 SCOPE OF THE PROJECT

In the current repository, the disaster classes used for training and evaluation are fire, storm, earthquake, and flood. Worth mentioning, the folder name `IOT/` is kept from earlier development when we were exploring live sensor integration, but the present environmental branch is trained on archived datasets rather than live field hardware.

The work completed in this project covers data preparation, branch-model training, fusion training, evaluation, explanation output, API serving, and the web interface. That's a lot of ground. Field deployment, agency integration, and testing on a naturally aligned real-world tri-modal dataset were outside the scope of the current implementation. Those would be the natural next steps though.

1.5 KEY CONTRIBUTIONS

The first contribution is the tri-modal fusion pipeline itself. Environmental metadata features, crisis-media features, and satellite features get combined inside a single decision layer instead of being processed as separate tasks. This is the core of

what makes the system different from just running three independent models side by side.

The second contribution is adaptive modality handling. The system learns which environmental sensor groups are relevant to a given case, and it still spits out a reasonable output when satellite data or environmental data is missing. It doesn't just break when something isn't available. It adapts and works with what it has. We noticed during testing that the graceful degradation was actually better than we expected.

The third contribution is a usable software layer built on top of the trained models. The repository includes explanation generation, a FastAPI server, background job handling, saved-case review, and a web interface. The entire system can be run end to end without additional setup. That was important to us because we wanted this to feel like a real application, not just a research experiment sitting in a notebook somewhere.

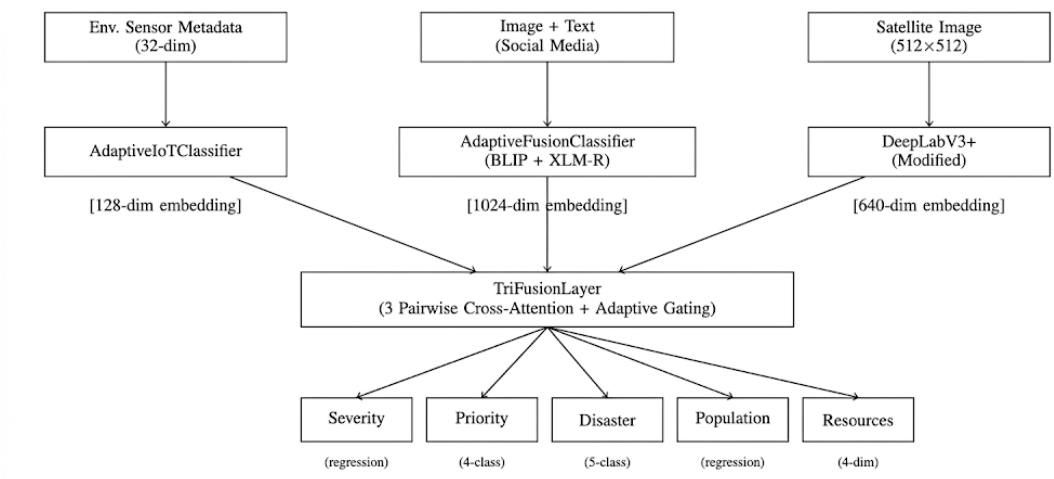

Figure 1.1 Overall architecture of the Multimodal Disaster Intelligence Platform showing the three upstream branches, the fusion layer, and the downstream decision heads.

1.6 ORGANIZATION OF THE REPORT

Chapter 2 covers related work. Chapter 3 describes the system structure and datasets. Chapter 4 presents the proposed method, and Chapter 5 deals with implementation details and the web application. Chapter 6 reports experimental results. Chapter 7 concludes with limitations and directions for future work.

CHAPTER 2

LITERATURE REVIEW

2.1 SOCIAL MEDIA FOR CRISIS UNDERSTANDING

Most of the early crisis analysis work was text-only. The systems would take in tweets or short messages or posts about some event, but even when photos were attached they'd only look at the text. Just the text. It's kind of surprising in hindsight because images carry so much useful information during a disaster. But that's how things were for a long time, a lot of the foundational work in crisis informatics was built entirely around NLP pipelines that ignored visual content altogether.

CrisisMMD basically proved that if you fuse image and text together you get better results than using either one alone, and we saw the same thing in our own pipeline when we ran the comparisons. That said, most of the multimodal work out there stops at classifying images and text from social media posts. It doesn't go further. Nobody is really combining that social media classification with satellite imagery of the actual damage on the ground, or with situational awareness data from environmental sensors. There's a real gap there.

2.2 SATELLITE DAMAGE ASSESSMENT

Satellite imagery is useful because it lets us see on-the-ground impact across large geographical areas all at once. Benchmarks like xBD and xView2 made building-damage segmentation a standardized task. The go-to architectures for this are DeepLab and U-Net. They've pretty much become the default choices for satellite damage assessment work and that's what we ended up using too.

Our satellite branch does two things in this architecture. It's not just a standalone segmenter, we also use it as a feature provider for the fusion layer. This distinction actually matters quite a bit because the pixel-level predictions don't need to be perfect across all damage classes as long as the satellite embedding that comes out is good enough to help the fused prediction downstream. It took us a while to realize that.

2.3 ENVIRONMENTAL AND SENSOR-BASED HAZARD MONITORING

Recording things like rainfall, pressure, wind speeds, and similar measurements for floods, storms, or earthquakes is nothing new. It's well established and has been for

decades. The useful thing about environmental data is that it can tell us about the physical characteristics of a disaster area before anyone even submits a public report or posts on social media about it.

The problem with a lot of existing hazard monitoring pipelines is that they only handle one task and they weight every sensor group equally. Which doesn't make a lot of sense when you think about it. Our environmental monitoring tree ended up taking a different approach where it nests features together and then decides which cluster should get more weight depending on what the input actually looks like. After trying a few approaches we settled on this grouped confidence mechanism.

2.4 MULTIMODAL FUSION AND CROSS-ATTENTION

The simplest multimodal models just concatenate the feature vectors at the end of each branch and call it a day. More recent work has introduced attention or cross-attention layers that let one input bias how another input is represented. The second approach turned out to be way more useful for disaster analysis. Image, text, and environmental inputs shouldn't necessarily carry the same weight for every type of disaster, a concatenation-only model has no mechanism to adjust that.

There are a handful of works that have tested fusion with three inputs, but multimodal research generally only handles two modalities at a time. We go further than that. The idea was to create three separate branches, apply cross-attention pairwise between the modalities, and then train a gating mechanism that learns how to pull them all together. We also handle missing modalities during training so that the system can still produce inferences even when fewer than three inputs are available. This matters a lot in real disaster scenarios where data arrives unpredictably and you can't just wait around for everything to show up.

2.5 EXPLAINABILITY IN HIGH-STAKES AI SYSTEMS

Just returning a label or a confidence score is rarely enough for a disaster response system. People actually using these tools want to know how the model reached its decision. Methods like Grad-CAM can visualize which part of an image was most important for a given prediction, that kind of visual evidence goes a long way toward building trust.

We go a step further by building explanation directly into the standard workflow. Heatmaps, class probabilities, modality weights, and sensor group weights are all displayed on the incident page right alongside the final prediction. The user doesn't have to go looking for them separately. We noticed during testing that having explanations immediately visible made a big difference in how people interpreted the results.

2.6 RESEARCH GAP

Most projects only address one subproblem of disaster analysis. Some build models for social media processing but don't combine that with satellite or sensor data. Satellite models might produce excellent segmentation results yet they can't describe those results in a way that's actually usable for decision-making. Sensor monitoring systems can chew through huge volumes of live sensor data but have no mechanism at all for handling text or imagery that arrives afterward. Everything ends up siloed and disconnected.

Projects that combine all three modalities are rare. And web or mobile applications that serve as clients to such systems are even rarer still. Rather than just stopping at model training and benchmarking on a test split, we provide tools for model training, a fusion method, result explainability, API serving, and a client web application. All within the same repository. That's what separates this project from a small-scale benchmarking effort.

2.7 COMPARATIVE ANALYSIS OF EXISTING DISASTER INTELLIGENCE SYSTEMS

Traditional disaster intelligence platforms work through data from a single modality, and each one has its own strengths and weaknesses. Social media systems focus on calls for help, humanitarian needs, and user-generated ground truth from the public, but they require frequent users and clean language to work well. Satellite systems have near-global spatial coverage and exclusive access to building damage metrics, but they suffer from revisit latency, cloud cover, and delays in image availability. Sensor systems ingest the physical phenomena produced by natural hazards. Rainfall rates, wind speeds, temperatures, earthquake metrics, and river gauge levels are all valuable, but on their own they tell us nothing about destroyed infrastructure or affected residents. That's a fundamental limitation.

DISCO builds on this prior work but differs because we're merging social media, satellite imagery, and geocoded sensor observations into one operational framework. We don't treat environmental metadata, crisis tweets, and satellite images as three separate products. Instead we learn a multimodal representation that informs downstream tasks like disaster type classification, priority and evacuation needs estimation, severity prediction, and resource-level decision making.

The reasoning behind this multimodal fusion is pretty straightforward honestly. Emergency response and recovery teams never rely on signals from just one modality. If we want ground truth from citizens in an affected area, we need social media. To understand the scope of a disaster, we need satellite imagery. Satellites can reveal building-level damage from space but social media tells us whether entire neighborhoods or cities are actually affected and what the situation looks like on the ground. Sensors from sources like NOAA give us predictions about a hurricane's wind field, but without overlaying Twitter data disaster teams can't visualize those winds in a people-focused context.

Each modality covers a different part of the disaster response problem, and single-modality systems break entirely when their data source fails. This is a big deal that doesn't get talked about enough. Satellites are useless until they've actually passed over the disaster region and collected new imagery. Social media can be oversaturated with off-topic messages or repeated calls for help that make it hard to extract useful signal. Sensors can detect when a flood is developing based on river conditions, well before there is any actual threat to people, which is useful but also limited. No system can perfectly handle missing modalities, but we can build a model that learns to work with different subsets of the available data. Our tri-modal architecture supports three fusion strategies. When all three data types are available, the network does tri-modal fusion by default. If social media becomes unavailable for a given disaster region we use the model's early fusion output to produce an answer from satellite imagery and sensor data alone.

This comparison also points to another benefit of the modal fusion layer. The way this works is by concatenating the output of each unimodal network, the model can attend to all modalities for any given input. The gated fusion layer lets the model decide adaptively how much weight each modality should receive. If satellite imagery is clear and available the model should learn to rely more on imagery features. Social media

becomes more useful when affected populations are actually able to report on their situation. When both satellites and social media are unavailable or noisy, environmental metadata is the best remaining window into what's happening with the crisis. We tested a bunch of different weighting schemes before landing on this approach.

In summary, while prior work has shown how to use multimodal disaster data, there's a gap in deploying it inside a practical system that is both explainable and aware of modality degradation. To our knowledge this is the first multimodal system that can flexibly combine heterogeneous disaster evidence within the time constraints that first responders actually face. Other models do model-level fusion but none of them predict across four disaster management tasks. We're also not aware of other multimodal models that include an explainability component or a dashboard interface for crowdsourcing recovery insights.

CHAPTER 3

SYSTEM DESIGN AND DATA PREPARATION

3.1 SYSTEM OVERVIEW

The system has three branches and a fusion stage. One branch handles environmental values, another takes in crisis image and text input, the third works through satellite imagery. Each branch spits out embeddings, and these get concatenated and passed into the tri-fusion layer.

Splitting the model into separate branches made development and debugging way simpler. We could add modules to each branch independently without breaking other components. And the serving layer only had to work with prepared vectors rather than having to parse raw input files every time. This was one of those design decisions that saved us a lot of headaches down the road.

3.2 PROPOSED SYSTEM ARCHITECTURE

The Multimodal Disaster Intelligence Platform is set up as a branch-based pipeline where each data source gets processed by its own specialized model before being combined into a single disaster assessment. We put the architecture together this way because relying on one source of evidence during an emergency just isn't reliable. In practice information arrives unevenly during a disaster. Environmental and seismic readings tend to be available early, social media reports come in continuously but with a lot of noise, and satellite imagery may be delayed by cloud cover, revisit schedules, or download constraints. To deal with all of this the system uses independent modality branches followed by an adaptive fusion layer that can handle whatever subset of data is actually available.

The first branch consumes environmental metadata. It accepts a 32-dimensional numeric feature vector made up of weather, storm, seismic, and hydrological indicators. Unlike other models we encountered during our literature review, these features are not fed in as a flat list of unconnected numbers. Instead they're grouped by their physical measurement type so the model can learn which feature family matters more for a given disaster type. Like, storm features should carry more weight when the disaster is a cyclone or heavy rainfall event, while seismic features should matter more for earthquakes. That's the basic intuition. After grouping the inputs are mapped to a hidden

representation space and weighted by the branch's confidence in each group. Compared to a simple dense classifier this setup gives us more interpretability because the per-group weights learned by the network can actually be inspected to see which sensor group contributed most to a given prediction. We spent a lot of time debugging this part because the confidence scores were initially collapsing to uniform values.

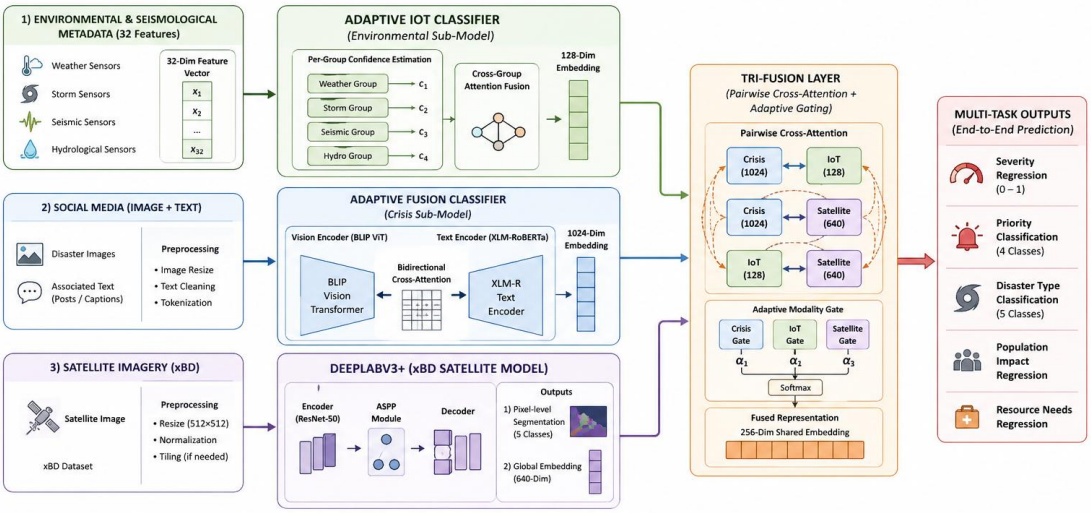

Figure 3.1 Overall architecture of the proposed Multimodal Disaster Intelligence Platform.

During real disaster events, people tend to share images, short summaries, emergency requests, or situation updates through social media and other public communication channels. This data is useful because it's firsthand observations from people on the ground, it can give strong signals about urgency, affected local regions, type of crisis, and human impact. On the other hand social media and crisis reports can be noisy, incomplete, or ambiguous, which is a real problem we had to deal with. To handle this we built two separate encoders: a vision encoder (ViT-B/32) for images and a language encoder for text. The text and image embeddings are projected into a shared space and then fused with cross-modal attention. The way cross attention works here is it lets the model capture relationships between what appears in the photo and what the text describes. A picture of flooding becomes way more informative when the accompanying text mentions trapped people, road closures, or the need for medical help.

Satellite imagery offers a much broader scope of information than any single social media post can. Satellites can reveal damage patterns across entire cities or affected regions, which is something no other data source can really do at that scale. For this branch we use the architecture from our previous work, a Modified

DeepLabV3+ trained on xBD-style satellite images for post-disaster damage assessment. That model was originally built to produce per-pixel segmentation masks of damaged buildings. But its role here is different, here it's meant to generate satellite-based embeddings representing the physical state of the Earth's surface after a disaster. These embeddings carry information about building damage that the fusion model can use to support its predictions, particularly when social media or crisis text isn't available for a given area.

Once each branch kicks out its embeddings, they get passed through a fusion network before a final prediction is made. The fusion layer is where the different modalities actually get combined and it was put together with pairwise cross attention and an adaptive gating mechanism. The cross attention between modality pairs gives the model a way to understand how two different pieces of evidence relate to each other. Satellite information about damaged buildings can be compared against the implied urgency of a crisis text message. Or environmental data showing heavy rainfall can be checked against damage evidence from images or text descriptions. Understanding how these different modalities relate is important for determining disaster severity, a hurricane with high rainfall might sound severe on its own but the predicted severity should also take into account whether there are people, buildings, roads, or city services actually affected in the area.

Adaptive gating is what lets each piece of evidence receive more or less importance during prediction based on what the other modalities are saying. Not all inputs will be equally useful for every prediction so this design allows each modality to basically signal how relevant its evidence is for the fused output. If the satellite imagery is old or out of date the model can learn to rely less on it. If social media text is vague and doesn't have useful keywords the model can learn to reduce its weight. If environmental metadata is missing entirely the model can compensate using only crisis and satellite information. By learning confidence values for each modality the system can handle partial-input situations where one or two modalities are unavailable. When all three signals are present the system works with full tri-modal input. If satellite imagery is unavailable, predictions can still be made from crisis and environmental data. If metadata is missing the crisis and satellite branches can still be used to estimate severity. In the worst case with both satellite and metadata unavailable, the model falls

back to image/text crisis data alone. We'd expect a drop in performance there and our ablation results confirm that.

The fused representation feeds into a handful of prediction heads that each handle a different task. These heads predict disaster type, severity score, priority level, impacted population, and resource needs indicators. We use multi-task prediction because emergency response decisions depend on more than just a single class label. A disaster intelligence platform shouldn't just categorize something as a flood, fire, storm, or earthquake and stop there, it should also help estimate urgency and the type of operations that might be needed in response. Sharing the fused representation across prediction heads lets the model learn a common disaster context while still cranking out task-specific outputs. This ended up working better than having completely separate classifiers for each task which is what we tried first.

The architecture also supports explainability, which we think is really important and maybe undervalued in this space. Beyond making predictions the model should be able to offer up evidence or justification for them. Since this system is intended for high-risk disaster intelligence, having unexplained black-box outputs wouldn't be acceptable for end users. The environmental branch can provide sensor-group confidence weights. The crisis branch allows image-text attention to be analyzed. The satellite branch can supply visual evidence of damage. Together the fused model can output both a prediction and the evidence behind it. This matters because disaster response decisions often need to be justified and reviewed and compared against on-the-ground knowledge from first responders.

Overall this architecture is intended for operational disaster intelligence rather than just benchmark classification. It does this by extracting predictions from three different types of evidence. Environmental metadata carries signals about physical hazards. Crisis images and text give a human-verified, local perspective on the disaster. Satellite imagery can show regional damage to specific areas. The tri-fusion layer pulls together these distinct pieces of information into a single operational representation. And fusion also lets the network operate even when one or two input streams are missing, which is important because in real disaster situations data can arrive late or in poor quality or sometimes not arrive at all.

3.3 MODALITIES USED IN THE PROJECT

The environmental branch takes in a feature vector with 32 numerical inputs, split evenly into groups of weather values, storm values, seismic values, and hydrological values. The crisis branch takes in one image and one associated text report. The satellite branch expects imagery downloaded from the xBD post-disaster satellite pipeline.

These modalities were selected because they describe different characteristics of disaster events. Environmental metadata captures conditions related to natural hazards. Social media gives insight into what people are reporting about a crisis. And satellite imagery shows physically damaged structures over wide areas. Each modality fills a gap that the others can't, that was the whole point of using three instead of two.

Table 3.1 Complementary role of each modality in the proposed platform

Modality	Primary strength	Typical weakness	Contribution inside fusion
Environmental metadata	Captures physical hazard signatures and contextual risk	Limited human impact detail	Improves hazard discrimination and context
Social media image-text	Fast public-facing situation updates	Noisy, incomplete, and credibility-sensitive	Adds on-the-ground impact evidence
Satellite imagery	Wide-area structural damage visibility	May arrive later and miss community context	Strong signal for severity and priority

3.4 ENVIRONMENTAL DATASET PREPARATION

Although the codebase refers to this modality as `IOT`, the environmental branch was actually trained on datasets obtained from APIs rather than live monitoring devices. We should probably clarify that since it causes confusion. The sources included in this repo are fire-weather records, tropical storm tracks, earthquake catalogs, and one dataset describing flood risk. Each dataset had to be reformatted to match the uniform structure expected by the model before training could begin and that reformatting was more work than we initially anticipated.

The environmental dataset contains a total of 63,527 preprocessed samples. The labels are fire, storm, earthquake, flood, and unknown. Each source dataset had different columns and scales and column-naming conventions, so reconciling all of that took a fair amount of preprocessing work. We ran into some issues with this at first because a few of the source datasets used inconsistent units.

Table 3.2 Environmental metadata sources used in the project

Source dataset	Mapped hazard class	Rows used
CA Wildfire weather and fire records	Fire	14,988
Historical Tropical Storm Tracks	Storm	59,228
Atlantic storm records	Storm	22,705
Global earthquake catalog	Earthquake	8,394
Iran earthquake catalog	Earthquake	52,268
Sri Lanka flood risk dataset	Flood	25,000

3.5 CRISIS AND SATELLITE DATASETS

The crisis branch uses CrisisMMD, which is a dataset of paired images and short text messages with humanitarian labels. It's a good fit for this project because our crisis branch is specifically set up to process image and text together, and CrisisMMD gives us exactly that kind of paired data.

The satellite branch uses xBD-based post-disaster imagery. In this project that branch is used both for damage understanding and for generating embeddings that feed into the fusion layer.

3.6 DATASET CHARACTERISTICS AND PREPROCESSING DETAILS

The datasets used across the three branches differ in structure, scale, and semantics. The environmental branch uses metadata tables. The crisis branch uses paired images and text. The satellite branch uses pre/post disaster imagery with damage labels. Because of this heterogeneity, preprocessing ended up being a pretty significant part of the system design, more than we expected honestly. A single preprocessing strategy couldn't be applied across all inputs so each modality required its own preparation pipeline before its output could be fed into the fusion model.

For the environmental branch the raw datasets contained numerical weather, storm, earthquake, and flood risk features. We transformed all of this into a fixed-length

32-dimensional input vector. The vector was divided into four groups: weather features, storm features, seismic features, and hydrological features. These groupings were intentional, we kept them on purpose so that physical meaning would be preserved inside the model. Encoding the representation this way lets the model learn which type of environmental evidence matters for each class of disaster rather than treating every feature as an independent cue with no context. We also applied controlled preprocessing of missing or unavailable values so the model would always have a full feature vector during both training and inference.

The crisis branch required paired image and text data. The image gives us visual evidence while the text adds semantic and contextual information. Text reports were cleaned to remove noise while keeping disaster-relevant meaning intact, images were resized and normalized to meet the requirements of the vision encoder. Since crisis posts can contain incomplete or ambiguous descriptions the text branch was not used on its own. Instead image and text embeddings were fused so the model could learn the relationships between what is shown visually and what is described in words. This turned out to be really important for getting good predictions, more so than we originally thought.

The satellite branch used post-disaster imagery for physical damage assessment. Satellite images were resized to a consistent input resolution before being passed into the modified DeepLabV3+ pipeline. The damage classes are no damage, minor damage, major damage, and destroyed, these were represented as segmentation labels. Although the satellite model can produce pixel-level damage maps its main role in the tri-modal system is actually to generate damage-aware embeddings. These embeddings give the fusion model structural evidence about physical damage that it couldn't get from the other branches.

Preprocessing also helped align the different modalities for fusion. Environmental features, crisis embeddings, and satellite embeddings naturally have different dimensions and meanings so you can't just concatenate them directly without things going wrong. Each branch first converts its input into a learned representation and these representations are then projected into compatible dimensions before the fusion layer works through them. This modular preparation lets each sub-model specialize in its own data type while still contributing to a shared disaster assessment.

3.7 FEATURE ENGINEERING STRATEGY

Environmental values are standardized to a single scale then divided into their respective groups. Inputs that include date or time-of-day information were sin/cos encoded to preserve their cyclic nature. Some inputs were also inverted so that higher numbers correspond to higher disaster significance where that made sense.

Inputs are grouped for another reason that becomes relevant later in the model. The fusion model is rigged up to weight each modality and potentially each group within a modality based on how useful it expects those inputs to be. If all weather, storm, seismic, and hydro values were just fed into a single downstream model as a flat vector there'd be no way to selectively boost or suppress certain groups of inputs. We would lose a lot of the interpretability that the grouped design gives us and that was something we wanted to preserve.

3.8 PROBLEM FORMULATION

Tri-fusion was set up as a multi-task learning problem. Rather than training the network to predict a single label it predicts disaster type, severity, priority, affected population, and resource needs from each sample. All of these come from the same fused representation.

Since the branches spit out embeddings of different sizes, a fully connected layer was used to project each branch's output into the same dimensional space before concatenation and input to the fusion layer. Pretty standard approach but it was necessary to make the fusion work properly.

CHAPTER 4

PROPOSED METHODOLOGY

4.1 ENVIRONMENTAL BRANCH: ADAPTIVEIOTCLASSIFIER

The environmental branch is built around four sensor groups. Each group gets encoded on its own and every 8-value group is mapped into a 128-dimensional hidden representation. That's it for the initial encoding step, nothing fancy.

Rather than treating every sensor group equally the branch has a `SensorConfidenceEstimator` attached to each group. The idea was to let the model figure out which group is actually more useful for whatever sample it's looking at right now, then apply that score when weighting the groups together. So instead of a flat equal weighting the model learns to pay more attention to the groups that matter for each specific case. This was one of the trickier parts to get working because the estimator needed enough signal from each group to produce meaningful confidence scores.

After the weighting step all four group embeddings go through cross-group attention. The pooled result from that is then used for disaster type, severity, and the other related outputs.

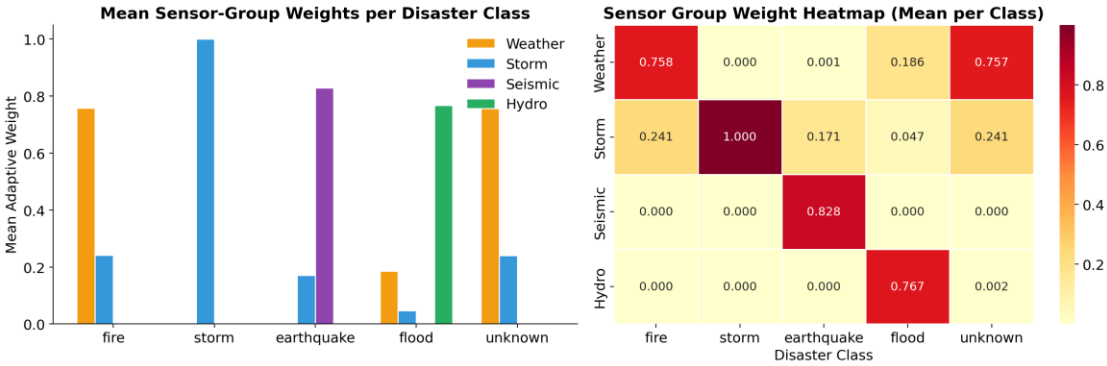

Figure 4.1 Learned sensor-group importance across disaster categories in the environmental branch. The model assigns high weight to the physically relevant feature families for each hazard type.

4.2 ENVIRONMENTAL FEATURE GROUPING AND CONFIDENCE WEIGHTING

Feature dependency is really central to how the environmental branch works. We had to think about this carefully and it took a few iterations to get right. Wildfire events correlate strongly with temperature, wind speed, and humidity, while storm events correlate more with wind speed, pressure, and precipitation instead. Seismic factors

form the basis for earthquake predictions. Flood hazard depends heavily on hydrological and precipitation inputs. The point is environmental model inputs shouldn't just be treated as flat, unstructured numeric vectors because these relationships between feature groups actually matter a lot for getting the predictions right.

The input is an 8x4 matrix where each eight-dimensional vector corresponds to weather, storm, seismic, or hydrological factors. Each of the four feature types passes through a separate encoder and becomes an embedding. The reason we use group-specific encoders is that the model needs to detect patterns within each group before any aggregation happens. Once the embeddings are extracted the model computes a confidence score for each feature group. These confidence scores act as adaptive weights that reflect how important a given feature group is for the current prediction, so the model can dynamically shift its focus depending on what kind of disaster event it's dealing with.

This also improves model interpretability, which we think is pretty important for a system like this. For instance it's straightforward to justify a storm prediction if the model's output relies heavily on high scores in the storm feature group. Same logic holds for earthquake prediction where the model assigns high scores to seismic factors. You could technically achieve the same classification results without confidence weighting, but the adaptive weights make the model's reasoning much easier to trace and explain.

Adaptive weights help with model performance in a more practical sense too, they reduce the negative impact of noise in model inputs. This turned out to be a bigger issue than we initially expected. Working with real environmental metadata introduces a bunch of problems, things like incomplete data, delayed inputs, and outright erroneous readings. In a fixed-weight model all four feature groups receive equal importance regardless of data quality which is obviously not ideal. Adaptive weighting sidesteps this by down-weighting unreliable groups automatically so noisy or missing data in one group doesn't drag down the whole prediction.

The output of the environment branch is a set of environmental embeddings. These embeddings summarize the physical hazard environment and feed into later

predictions. They aren't final output values on their own, they're passed into the tri-fusion layer along with crisis media and satellite embeddings.

4.3 CRISIS BRANCH: ADAPTIVEFUSIONCLASSIFIER

The crisis module takes an image and a textual input drawn from social media posts. The image input goes into a BLIP vision encoder. The text input is processed by XLM-RoBERTa. Both outputs are projected into a shared space before the two inputs are fused together.

Confidence for the image and text modules is learned separately since each component has different utility depending on the case. Sometimes the image is more useful, sometimes the text is. It really depends on the quality and content of each input.

Attention from images to texts and vice versa takes place in the module. The attended embeddings are concatenated and passed further as the crisis embedding.

4.4 IMAGE-TEXT REPRESENTATION IN THE CRISIS BRANCH

The crisis branch encodes human-generated reports that contain both visual and textual descriptions. We chose this modality because social media posts and disaster worker reports tend to provide the earliest human-level signals during a disaster. A brief text report might mention impassable roads, injured civilians, damaged homes, or urgent rescue needs. An image might show flooding, fire, collapsed structures, or crowd density. But here's the thing, either description can be incomplete on its own. An image could show standing water without indicating whether people are trapped, a text report could describe major damage without conveying the actual scale of destruction visually. So this branch fuses both representations to get a more complete picture and we found this fusion to be really important during our evaluation.
The text information is encoded using a language encoder that captures semantic content from the public message. Images are embedded using a vision encoder that pulls out salient features from the visual content including objects in the scene, structural composition, visible damage, and other relevant visual patterns. The image and text embeddings are then projected into a shared representation space since the raw embeddings from each encoder have different feature formats. After projection the model can compute multimodal similarity between the two modalities and learn relationships where one signal reinforces the other.

We use cross-modal attention mechanisms to couple the image and text representations. The way this works is image-to-text attention lets the model understand which words in the description are most relevant to what appears in the image, text-to-image attention lets the model focus on visual features that support or contradict claims in the text. Allowing each representation to influence the other produces inter-modality relationships that are stronger than what you'd get from simple concatenation. Like if a report says there are collapsed buildings nearby and the image actually shows structural damage, the joint representation ends up being much stronger as a result.

This branch also produces confidence-aware representations which is important because not all image/text pairs are equally informative. Some images may be blurry, unrelated, or taken under poor conditions. A lot of text reports are short, emotional, or not actually about damage at all. Learning how much each piece of information should contribute to the final output makes the model more robust to this kind of uncertainty. Since social media reports from the field are noisy but still valuable we want the model to handle that variability rather than just crashing or giving bad predictions when inputs are messy. The final crisis embedding therefore contains information about the message itself and also about its perceived reliability. We spent a lot of time getting this right.

Within the full system this branch pulls out local information about human impact. Environmental metadata can tell us about hazard conditions and satellite imagery can show large-scale regional damage. But reports from people on the ground convey urgency, identify victims who need help, and provide first-hand accounts that the other modalities simply can't capture. This is why the crisis branch is so important to our prediction and planning pipeline.

4.5 SATELLITE BRANCH: MODIFIED DEEPLABV3+

The satellite branch uses a modified DeepLabV3+ model. In our project it's used not only for segmentation but also to produce an embedding for the fusion layer. That dual role ended up being quite useful.

The branch outputs both damage-related spatial information and a compact scene representation. The satellite embedding size used in fusion is 640.

Experimental results show that this branch still has room for improvement on minority damage classes. But it contributes strongly to the final fused prediction even so, which surprised us a bit.

4.6 TRI-FUSION LAYER

The fusion layer first projects the environmental, crisis, and satellite embeddings into a single shared hidden size. This projection is necessary since the three branches don't produce vectors of the same dimensionality and we need them in a common space before any cross-modal interaction can happen.

After projection, the model runs three pairwise cross-attention steps — crisis with environment, crisis with satellite, environment with satellite. We tried a few different orderings here before settling on this one.

A modality gate then weights the branches before the final shared output gets assembled. If a modality is missing, the model falls back to a substitute representation and masks that branch out during gating. This was important for us because in actual disaster scenarios you really can't guarantee all three data sources show up. Sometimes you only get two, sometimes one, and the system has to deal with that.

Table 4.1 Core configuration of the tri-fusion model

Parameter	Value
Fusion model	TriFusionLayer with pairwise cross-attention and adaptive gating
Projected hidden dimension	256
Crisis embedding size	1024
Environmental embedding size	128
Satellite embedding size	640
Total parameters	3,163,602
Training split	10,886 train / 2,722 validation
Modality dropout	30% on environmental and satellite branches
Optimizer	AdamW with cosine annealing
Best validation loss	0.0288 at epoch 15

4.7 MULTI-TASK PREDICTION HEADS

The shared fusion output feeds into a handful of prediction heads. These heads spit out values for priority, disaster type, severity, population impact, and resource demand. It took us a while to get the head dimensions right.

We went with this setup because the project isn't just a hazard classification task. The final application also needs urgency and support-related outputs, which is why we have a few different prediction heads rather than just one. And honestly a single head wouldn't have been enough for what we needed.

4.8 EXPLAINABILITY AND USER-FACING REASONING

After prediction, the system also kicks out explanation outputs. For image-based inputs, Grad-CAM style visual outputs show which regions were important to the decision. We spent a decent amount of time debugging this part because the heatmaps kept coming out blank at first.

The project also uses fallback logic so that explanation output is still available even when one preferred path doesn't work for a given encoder or input. We didn't want a situation where the system just hands back nothing, no explanation at all.

4.9 EXPLAINABILITY WORKFLOW

Explainability is another requirement for the proposed disaster intelligence system, and honestly we think it's one of the more important ones. The output of a disaster prediction model can directly affect emergency response decisions. The model's prediction alone isn't enough to explain why a particular class or priority was picked. An explanation mechanism is needed so that operators can actually see why a specific result was produced. Basically the model should include explainability at each level of the disaster prediction pipeline, not just at the final output. We noticed during testing that even a simple confidence bar made operators way more comfortable with the results.

At the environmental level, explainability relates to the importance of different sensor groups for a given prediction. The environmental input consists of weather, storm, seismic, and hydrological sensor data, and what we need the explanation to show is which of these groups mattered most. This is necessary for linking predictions to actual physical phenomena. A flood prediction for example should point to hydrological features being important. Earthquake predictions should correlate with seismic features. If those connections aren't visible in the explanation then something is probably wrong with the model, that was our basic sanity check.

At the crisis-media level, the explanation should address how well the image and text align with each other, because visual and textual evidence really need to be presented and analyzed together. If the image shows signs of damage and the description contains urgent requests for help, the explanation should reflect that alignment. But if one modality lacks relevant information while the other provides evidence, the explanation should note that too. This kind of joint explanation gives

additional insight into how the decision was reached. After trying a few approaches to visualize this, we settled on a side-by-side layout.

At the satellite level, explainability should refer to physical damage. Damage can be assessed at both the building level and the regional level. Although the satellite branch is used within the fused model primarily to crank out an embedding, the underlying damage information is still valuable for review. It lets operators check whether the predicted severity lines up with visible evidence. This matters most when the model predicts high priority since that's when wrong predictions could have the worst consequences, and you really don't want to get those cases wrong.

Finally, explainability is also needed at the tri-fusion layer. This is arguably the trickiest part to explain. Because the tri-fusion layer involves interaction between three modalities, the explanation of the fused model should include the importance of each modality and how it contributed to the predicted class or priority. For instance a high-priority prediction might be explained by social media indicating affected people, satellite images showing structural damage, and environmental metadata indicating flood conditions. All pointing in the same direction. We spent a lot of time debugging this part to make sure the contribution weights actually made sense visually.

The project also includes a fallback approach for explainability, which we think is pretty critical. In practice not every explanation method works for every model architecture. Gradient-based methods may fail when gradients are weak, attention maps may not be available for certain components, and some inputs may simply be missing. To handle this, the explanation pipeline degrades gracefully. If the preferred explanation method is unavailable the system falls back to simpler evidence summaries — confidence scores, modality availability, prediction probabilities, input-level indicators. The idea is that the user always receives some form of reasoning rather than just an unexplained output. In hindsight, we should have built the fallback system earlier because we kept running into edge cases.

This explainability workflow makes the system more practical for real-world disaster use. We're not trying to claim the model explanation is perfect, far from it actually. But the goal is to give operators enough evidence for human review. In emergency management AI predictions should assist decision-making, not replace human judgment. By showing modality contributions and evidence summaries, the

platform helps operators interpret results, compare them with field knowledge, and decide whether additional verification is needed. That's ultimately what matters.

CHAPTER 5

IMPLEMENTATION AND SYSTEM INTEGRATION

5.1 REPOSITORY ORGANIZATION

The repository is split by major component. `IOT/` holds the environmental branch code, `crisis/` holds the social-media branch code, and `XBD/` has the satellite notebooks and data files. The `fusion/` folder is where the integration code lives — the FastAPI server, explanation utilities, and the web application are all in there. Pretty standard layout honestly.

The repo structure pretty much mirrors how we actually built the project. Each branch was trained and validated on its own first. Only after that did we stitch them together through the inference pipeline. This was one of those decisions that seemed obvious in retrospect but we debated it for a while.

5.2 TRAINING WORKFLOW

Training was done in stages. The environmental branch came first, then crisis, then satellite. The idea was to finish training and validation for each branch before moving on. Once all three were done we trained the tri-fusion layer on top of those persisted outputs. So it was very much a sequential thing.

This staged approach kept things smaller and easier to work with. Debugging one branch at a time was far simpler than trying to train the whole tri-modal system from scratch in a single pass, which would have been an absolute nightmare to troubleshoot. We learned this the hard way after one failed attempt at training everything together.

5.3 INFERENCE PIPELINE

At inference time, the system takes in a crisis-related image and text. It can also accept satellite data and environmental values if they happen to be available. What happens next is the server activates only the branches that have inputs and mashes their outputs together in a fusion layer to produce the final result. So it doesn't just crash if something is missing, it works with whatever it gets.

The core serving logic lives in fusion/server.py. The system supports both real-time analysis and asynchronous processing through background jobs using the /analysis/jobs endpoint. Uploaded files get temporarily stored on the server. Job status

is tracked so users can check results later, whether the job has finished or failed. We added the async job support pretty late in development but it turned out to be essential.

Table 5.1 Major implementation components in the repository

Path	Role in the system
IOT/train_iot.py	Training pipeline for the environmental metadata classifier
IOT/evaluate_iot.py	Evaluation scripts and metric generation for the environmental branch
crisis/train_crisis_code.ipynb	Training workflow for the social-media image-text model
XBD/final-xbd-deeplabv3plus.ipynb	Satellite damage model training workflow
fusion/train_tri_fusion.py	Training script for the tri-modal fusion layer
fusion/pipeline.py	End-to-end prediction assembly across modalities
fusion/server.py	FastAPI application exposing model inference
fusion/static/	Web dashboard pages, styles, and JavaScript

Table 1.2 Main web routes and their implementation roles

Route	Primary function	Implementation detail
/overview	Landing dashboard for latest system state	Summarizes current alert, detected hazard, priority, latest update, and recent saved runs
/analysis	Case submission workspace	Collects crisis image, report text, optional satellite image, and optional environmental values before scheduling analysis
/incident	Detailed fused result page	Displays alert summary, crisis probabilities, modality weights, sensor weights, resource demand, and Grad-CAM outputs
/iot-monitor	Environmental branch inspection page	Shows hazard-specific IoT outputs such as fire probability, storm category proxy, earthquake magnitude proxy, flood risk, and sensor-group weights
/reports	Saved history and export page	Reads stored incident runs from browser history and exports them as JSON for later review

5.4 DASHBOARD AND HUMAN INTERACTION LAYER

The repository includes a 5-page dashboard located at fusion/static/. The pages are: overview (index.html), analysis (analysis.html), incident details, IoT monitor, and reports (reports.html). That's pretty much the full frontend, nothing fancy.

The client stores history in the browser through `fusion/static/js/store.js`. Because of that a user can reopen saved cases and export stored report history from the reports page. This turned out to be really useful during testing, we didn't originally plan for it to be that central.

5.5 WEB APPLICATION WORKFLOW

The web application workflow is fairly straightforward. Users start on the overview page, open the new-analysis form, submit their case, and then view results on the incident page. Nothing complicated there.

The analysis form can send the request as a background job through `/analysis/jobs`. If the job starts processing the incident page shows a loading indicator until results are ready. After that the saved job can be loaded like any other incident, so there's no difference in the user experience between a quick result and a background-processed one. We tested this with a bunch of different file sizes to make sure.

On the incident page, we show the fused output with probability, modality weights, sensor weights, and Grad-CAM heatmaps. The reports page lists past runs and lets users download them. Pretty self-explanatory.

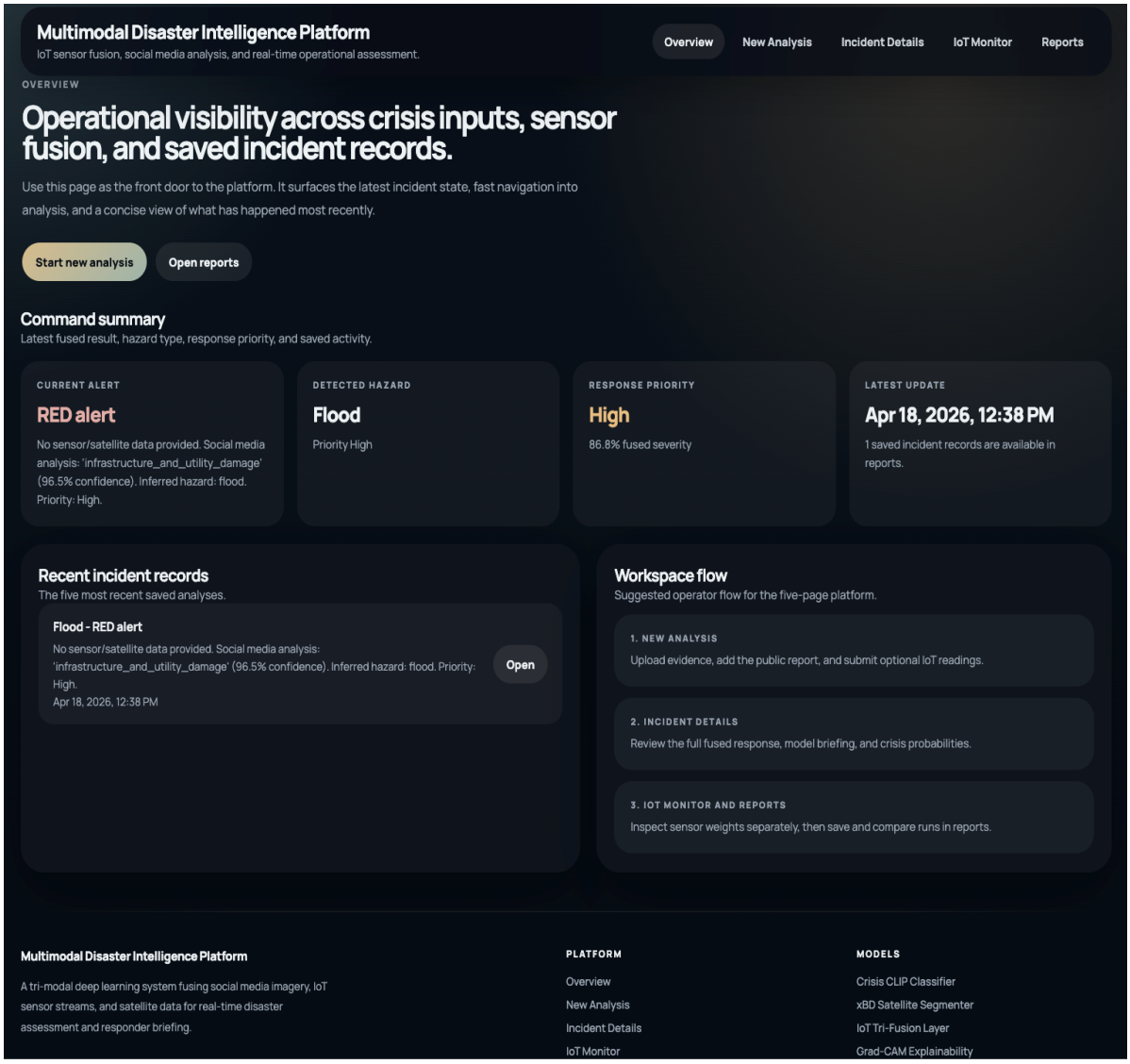

Figure 5.1 Overview page of the web application. It summarizes the latest alert state, detected hazard, response priority, recent incident records, and the recommended navigation flow across the platform.

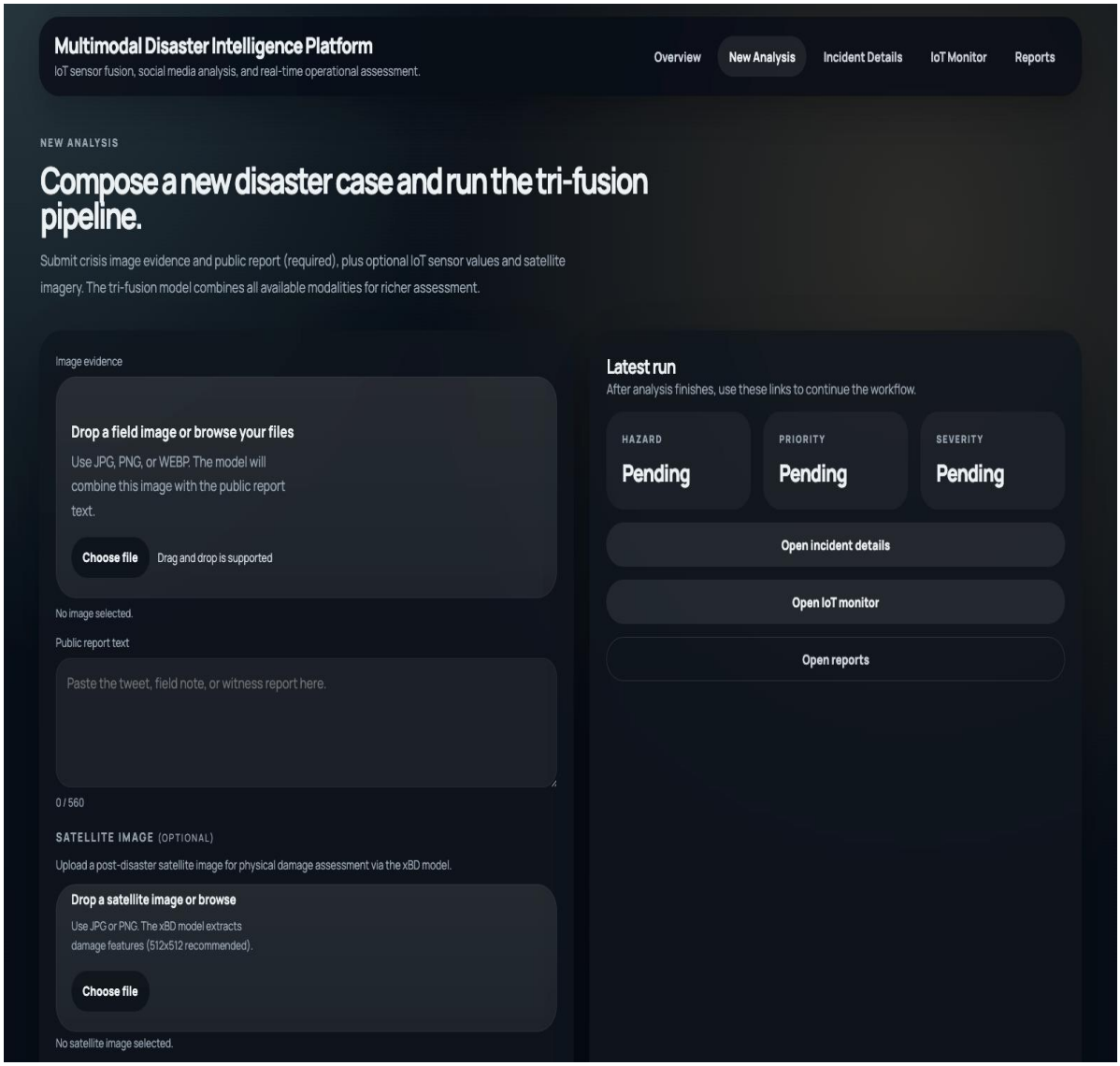

Figure 5.2 New-analysis page, first section. The operator can upload crisis imagery, enter the public report text, and optionally add satellite evidence before running the fused analysis.

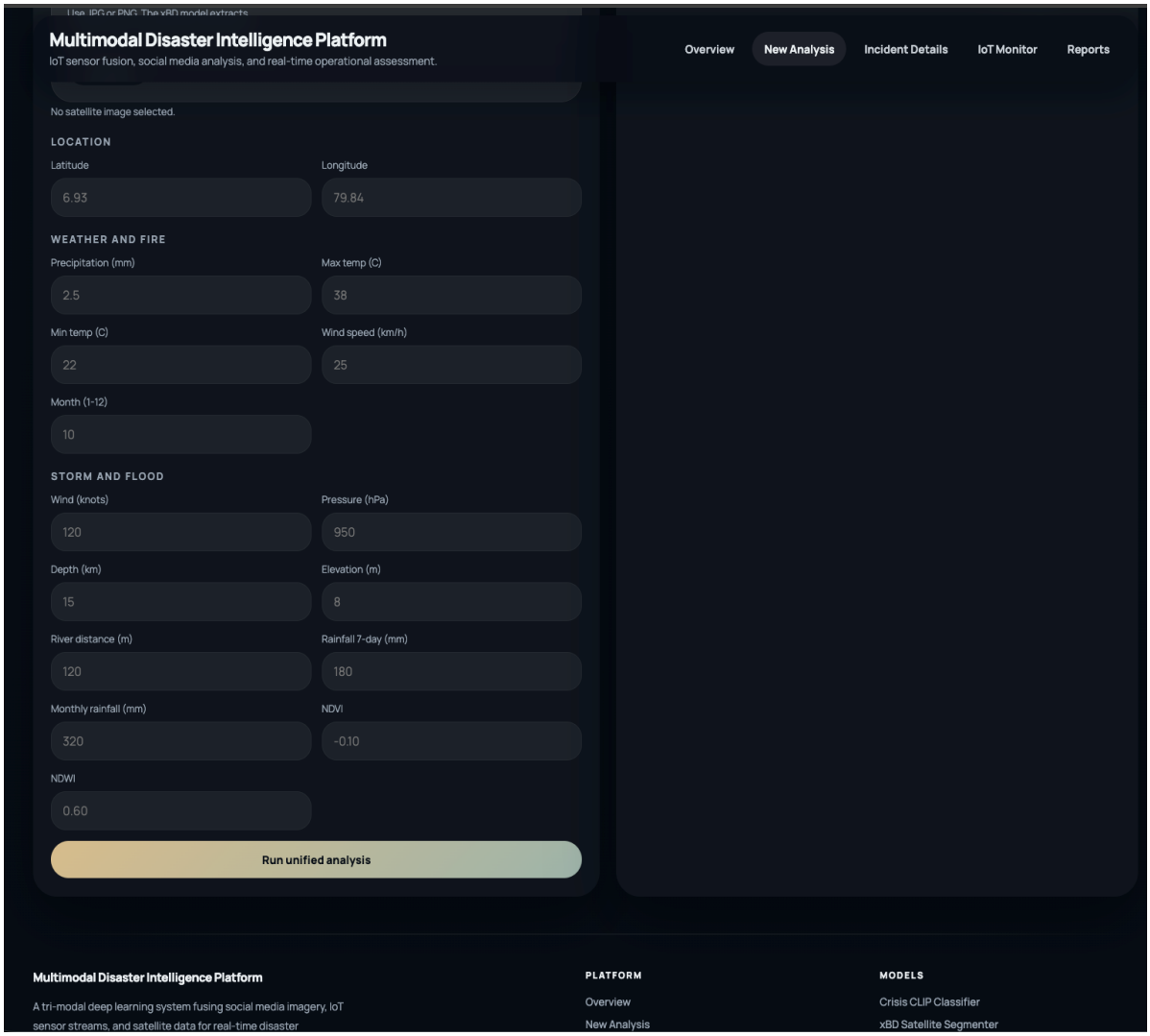

Figure 2.3 New-analysis page, second section. The form accepts geographic context and environmental measurements so the same workflow can combine crisis, satellite, and environmental inputs in one submission.

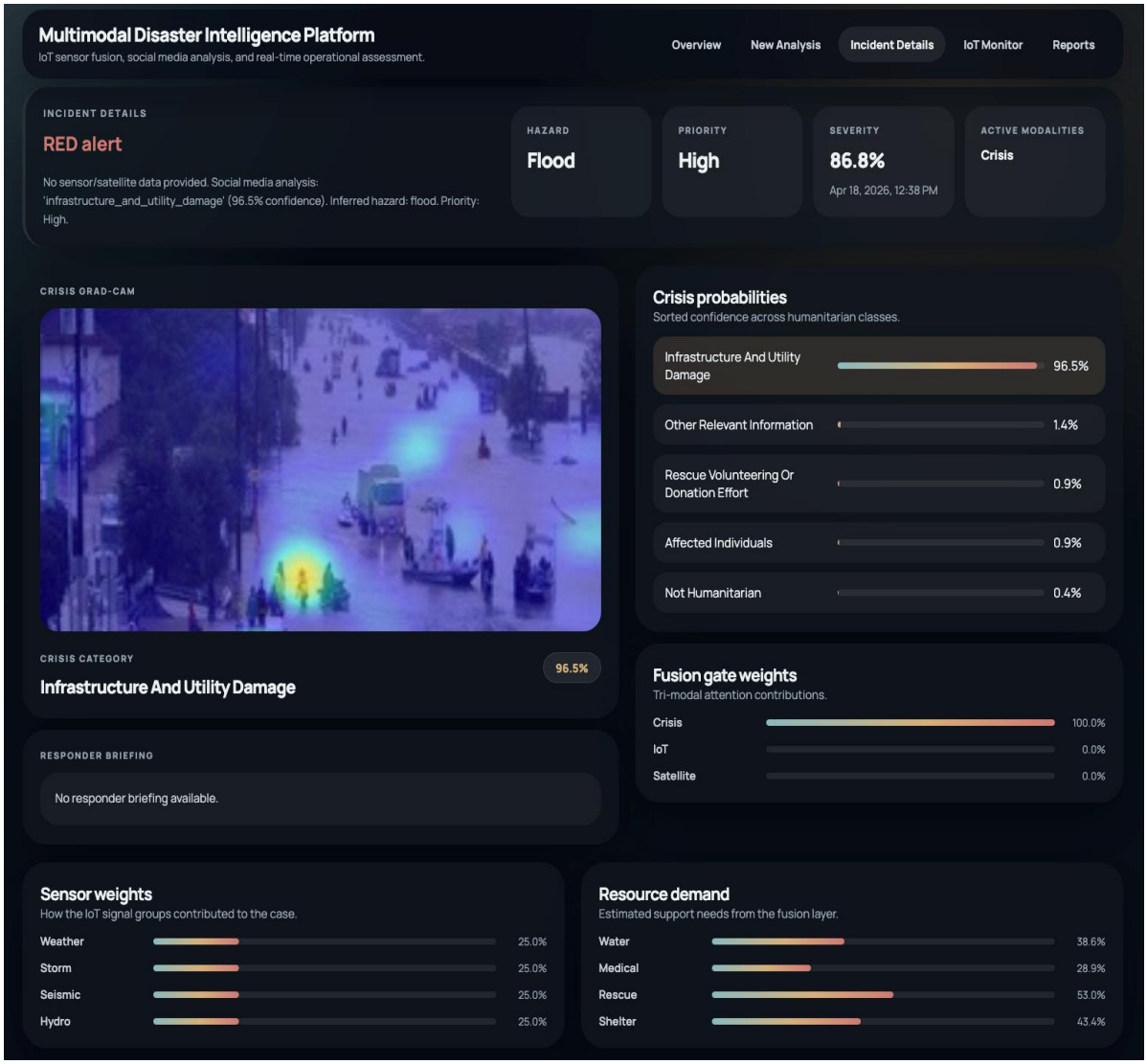

Figure 5.4 Incident-details page. This screen presents the fused alert level, hazard category, severity, modality contributions, crisis probabilities, sensor weights, and estimated resource demand for the active case.

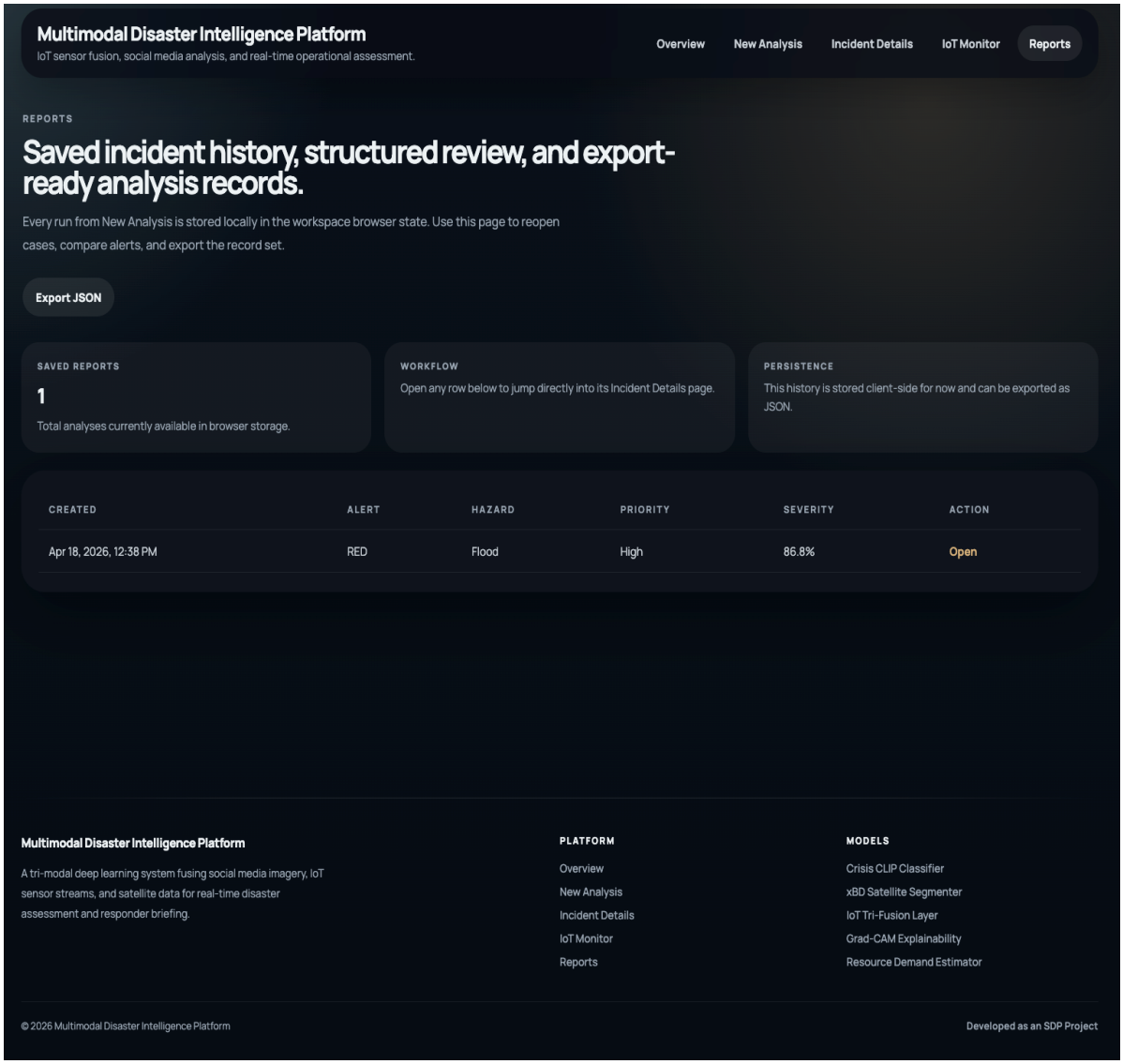

Figure 5.5 Reports page. Saved incident runs can be reviewed later, compared, and exported, which makes the application useful beyond one-time inference.

5.6 ENGINEERING TRADE-OFFS

Some parts of the project were trained under limited hardware conditions. This was especially true for the xBD and fusion stages. That affected how far we could push the experiments, we couldn't really do as many runs as we wanted to. It was frustrating but we worked with what we had.

The codebase also keeps the legacy IoT branch name. While the current environmental branch trained on historical data, we didn't rename IoT throughout the pipeline and associated files. So that's just something to be aware of when reading through the code, it's a naming leftover.

For the web layer, we went with browser-side history storage and a lightweight server-side job store instead of a full database. It's simpler and it worked well enough for our purposes. A proper database would be overkill for a prototype like this.

5.7 BACKEND API AND DEPLOYMENT WORKFLOW

Our deployment has a backend layer that connects trained models to the frontend dashboard. This system is meant to be more than just training scripts. We wanted to build an interactive disaster intelligence application where a user can upload evidence, submit a new report, run inference, and view results. So we put together the backend as the pipeline that links models to actual use cases. That's basically the core idea behind the whole architecture, and it's what makes this more than just a bunch of notebooks.

The user flow starts with creating a new inference job from the dashboard. A disaster case can contain crisis imagery, associated public report text, environmental values (optional), and satellite imagery (optional). The backend checks which inputs are available and formats them for the corresponding model branches. If a modality is missing the backend doesn't halt, it just flags that modality as unavailable and lets the fusion pipeline make predictions with whatever remains. That's how we handle graceful degradation. The system works with what it has and doesn't complain about what it doesn't.

Each model branch has its own preprocessing requirement. The environment branch expects a vector of numerical values with features in the correct order. The crisis branch expects an image and text pair, which get converted to tensors and tokenized text inputs. The satellite branch expects a post-disaster image prepared at the required spatial resolution. For all of these the backend is basically the interface that converts

frontend inputs into model-ready formats. This was one of the trickier parts to get working because each branch wanted its data in a slightly different shape.

Once inputs are formatted the backend runs each model component and collects outputs. Predictions from each branch go into the fusion stage, which then spits out final predictions: disaster type, severity, priority, predicted population affected, and resource estimates. Model outputs are returned to the frontend in a format the dashboard can display. The same response object can also carry explanation information — which modalities were available, confidence values, summary evidence, that sort of thing.

We kept the deployment workflow lightweight on purpose. There's no need for a large enterprise system here. The local directory can run on a single machine or a small server for demonstrations and evaluations. We found it useful to test the entire pipeline end to end while developing, it really helped catch bugs early. The frontend talks to the exposed dashboard endpoints. The backend handles loading models, running inference, and generating responses. This separation of concerns also makes the system extensible — later we could swap local model weights for cloud-hosted models, plug in live sensor streams, or log case history to a database. But for now this setup works.

As mentioned earlier the model pipeline is modular. Each component (environmental, crisis, satellite, fusion) is separated to reduce dependencies. The idea was that updating one component shouldn't require rewriting large portions of code. For instance we could improve the satellite model with a stronger segmentation backbone, or replace the crisis text encoder with a better language model, and it wouldn't break the rest of the pipeline. Discrete models also help in a domain like disaster intelligence because these components evolve quickly. Real-world requirements shift over time and you need to be able to swap things out.

Repeatability was another goal behind the backend design. We organized preprocessing, model execution, and output parsing so that it's easy to reproduce predictions or debug errors. If we see an unexpected output we can check what modalities were available, which branch gave low confidence, or whether the error happened during preprocessing or model inference. Building a system where we could trace failures mattered to us. It really mattered, given that the application deals with

disaster response where mistakes have real consequences. We noticed during testing that traceability saved us hours of debugging time.

CHAPTER 6

EXPERIMENTAL RESULTS AND DISCUSSION

6.1 EVALUATION STRATEGY

The evaluation follows two stages. First each branch model is tested on its own task, so we can see how well each part works independently. The image processing, text processing, etc. are all evaluated separately before we look at the combined system. This seemed like the most logical way to do it.

Then the full system is tested using an ablation approach where inputs are added or removed one at a time. This lets us observe how each modality affects the final output. We went back and forth on the ablation design before landing on this one.

Together these two stages show the contribution of each branch. What this means is we can see exactly how the tri-modal result changes when a particular input is included or left out. And that's really the whole point of the evaluation.

6.2 ENVIRONMENTAL BRANCH RESULTS

Based on the saved evaluation outputs, the environmental classifier reaches 97.64% overall accuracy, a macro F1 near 0.90, and a macro-ROC AUC above 0.996. Storm, earthquake, and flood are separated clearly. Fire and unknown are harder to distinguish though which makes sense because they share some overlapping feature ranges. We expected this going in.

The severity head reports a mean absolute error close to 0.045 and an R-squared value around 0.745.

The sensor-group weights behave as expected. Storm values dominate storm cases. Seismic values dominate earthquake cases. Hydrological values dominate flood cases. Pretty much what you'd want to see, no surprises there.

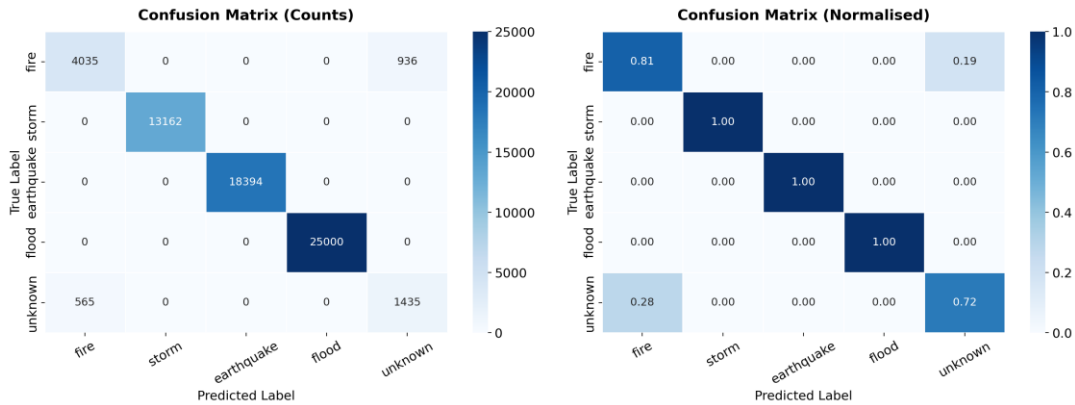

Figure 6.1 Confusion matrix for the environmental metadata classifier. Storm, earthquake, and flood classes are strongly separated, while fire and unknown show the main overlap.

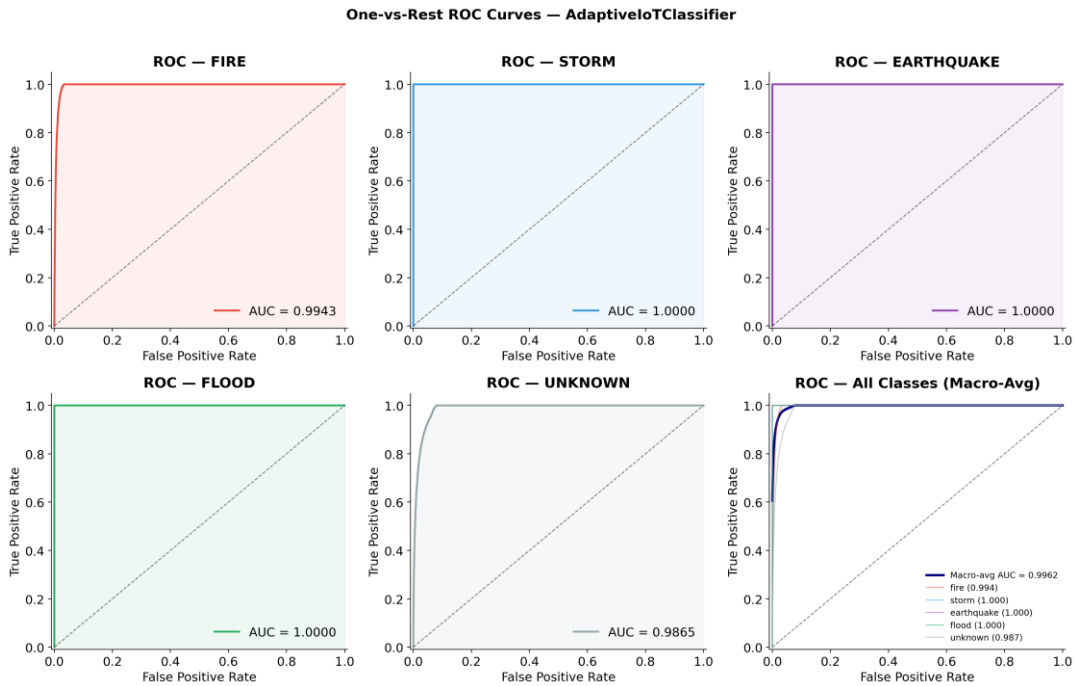

Figure 6.2 Per-class ROC curves for the environmental metadata classifier, showing strong class separation across most hazard categories.

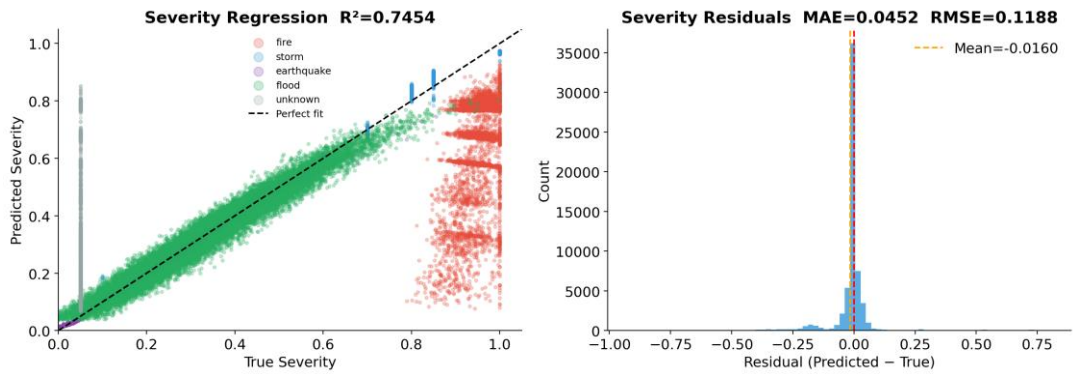

Figure 6.3 Severity regression analysis for the environmental branch, including agreement between predicted and target severity values and the residual pattern.

6.3 CRISIS BRANCH RESULTS

The crisis-media model pulls off 86.39% test accuracy and 87.48% validation F1 on the humanitarian classification task. This is a solid result given that the branch is also optimized to produce useful fusion embeddings, not just standalone accuracy. Looking at the per-class results the model does especially well on infrastructure damage, not-humanitarian content, and rescue-or-donation categories.

The weakest class is affected individuals. This isn't surprising given its lower support and the subtle overlap it has with other humanitarian categories. Even so the overall ROC behavior is good. And the training history shows that the dual-modality weighting stays balanced rather than collapsing to either image or text, which was something we were genuinely worried about early on. It took a few training runs to confirm this wasn't happening.

Table 2.1 Crisis branch per-class performance on the humanitarian task

Class	Precision	Recall	F1 score	AUC
Affected individuals	0.36	0.56	0.43	0.96
Infrastructure damage	0.87	0.90	0.88	0.98
Not humanitarian	0.90	0.87	0.89	0.96
Other relevant information	0.85	0.86	0.86	0.97
Rescue, volunteering, or donation effort	0.79	0.83	0.81	0.98
Macro average	-	-	0.774	0.97

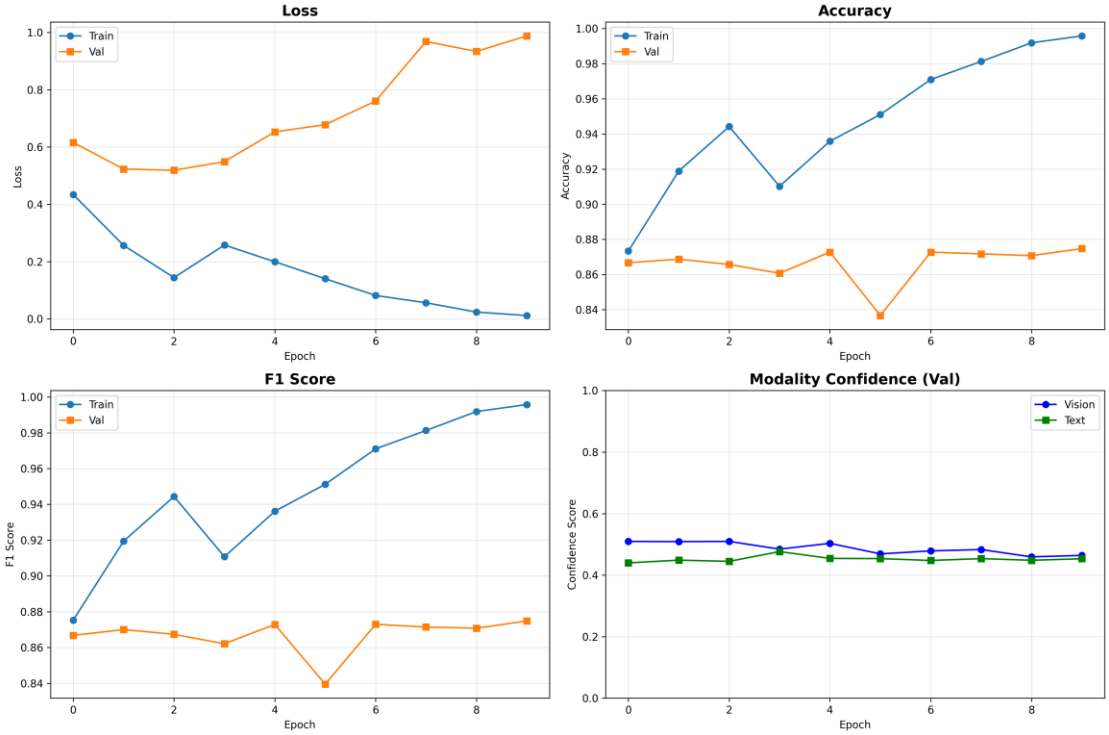

Figure 6.4 Training behavior of the crisis-media branch, showing stable validation performance and balanced confidence between visual and textual streams.

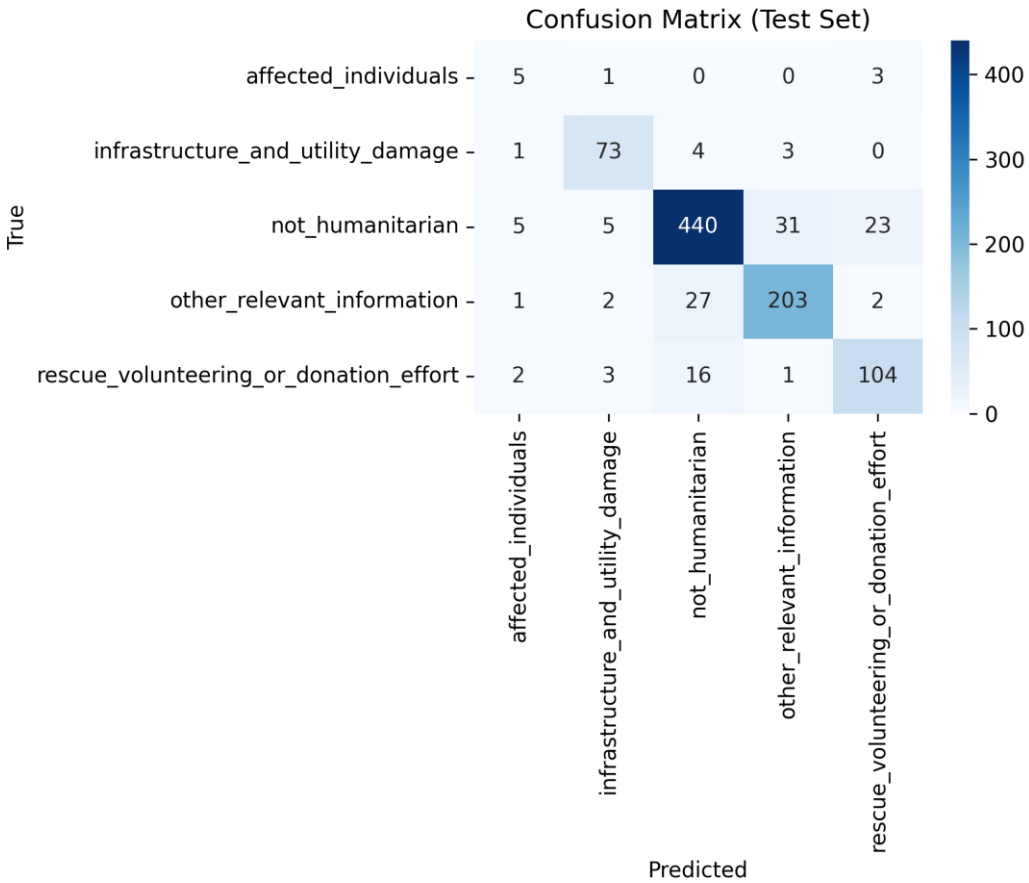

Figure 6.5 Confusion matrix for the crisis-media branch on the humanitarian classification task.

6.4 SATELLITE BRANCH RESULTS

The satellite branch reaches a best validation mean IoU of 0.4276 with mean F1 of 0.5221 in the checked-in experiments. The no-damage class performs very well while the minority damage classes are harder. This gap is expected given the class imbalance typical of xBD-style datasets, it's a well-known challenge in this area.

From a standalone segmentation perspective there's still room for improvement. But the more relevant question for this project is whether the satellite embedding improves the final tri-modal system. And the fusion ablation clearly shows that it does. Satellite evidence turned out to be the single strongest contributor to improved priority and severity prediction in the combined model. We honestly didn't expect going in that satellite would matter this much. That was a genuine surprise.

Table 6.2 Satellite branch class-wise validation performance

Damage class	IoU	F1 score
No damage	0.9774	0.9885
Minor damage	0.1595	0.2662
Major damage	0.2647	0.3936
Destroyed	0.3087	0.4402
Mean	0.4276	0.5221

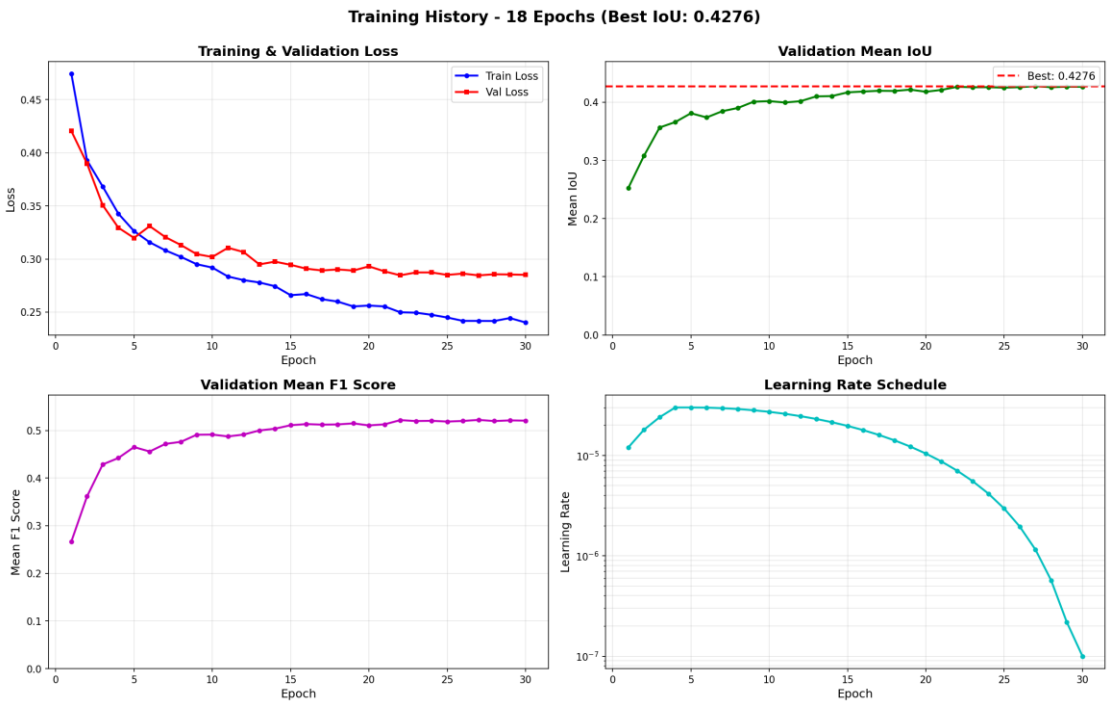

Figure 6.6 Training history of the satellite branch, including loss, mean IoU progression, and learning-rate scheduling.

6.5 TRI-FUSION ABLATION RESULTS

The ablation study shows what each modality adds to the final system. Crisis-only reaches 68.96% priority accuracy. Adding environmental data raises that to 72.37%. Adding satellite data raises it to 98.75%. The jump from environmental to satellite is massive, way bigger than we anticipated.

The full tri-modal setting reaches 99.41% priority accuracy. Severity mean absolute error drops from 0.1165 in crisis-only mode to 0.0398 in the full setting. So basically every modality we add improves things but the satellite data makes the biggest difference by far.

Across these settings results improve as more modalities are added. In the saved outputs the satellite branch provides the largest single gain. We already discussed this in the previous paragraph but it bears repeating because it's really the key finding.

Table 6.3 Ablation study across modality configurations

Configuration	Severity MAE	Priority accuracy	Disaster accuracy	Population MAE	Resource MAE
Crisis only	0.1165	68.96%	100.00%	0.0877	0.0468
Crisis + environmental metadata	0.0989	72.37%	100.00%	0.0691	0.0393
Crisis + satellite	0.0415	98.75%	100.00%	0.0274	0.0164
Crisis + environmental metadata + satellite	0.0398	99.41%	100.00%	0.0248	0.0161

6.6 ROBUSTNESS AND PRACTICAL OBSERVATIONS

Repository notes also include noise-injection tests for the environmental branch. Under moderate Gaussian noise the results degrade gradually rather than failing all at once, which is good behavior for a real-world system. We were relieved to see that honestly.

One other observation from the saved experiments is that disaster-type accuracy is already saturated across the fusion settings. Because of that severity and priority ended up being the more informative metrics for comparing configurations. So those are the numbers we focus on most.

6.7 LIMITATIONS

There are a handful of limitations worth noting. The environmental branch is trained on archival datasets rather than live field devices. The tri-modal fusion stage also uses synthetic pairing because a naturally aligned dataset wasn't available in the repository. This is a real limitation and we want to be upfront about it rather than burying it.

The satellite branch still underperforms on the minority damage classes compared to current state-of-the-art segmentation systems. Some of the training was also done under limited hardware conditions which didn't help. We would have liked more GPU time but that wasn't in the cards.

6.8 ERROR ANALYSIS AND CASE-LEVEL INTERPRETATION

The fusion model performs well on our experiments but it's worth looking more closely at where errors still happen. High overall accuracy on disaster intelligence data doesn't tell the full story. The operational cost of a few misclassified high- or low-priority predictions can be large, in a disaster scenario that really matters. So it helps to look at the system's behavior at the case level, modality level, and class level to find weaknesses. We did this manually for a bunch of cases.

In the environmental branch errors most likely occur when different disasters share similar numerical ranges. It can be rainy during both storms and floods. Wind speeds can be high during wildfires (because of fire spread conditions) and storms. The way this works is environmental metadata on its own could misclassify hazard classes that share similar physical phenomena. Confidence weighting reduces this problem to a degree but it can't fully distinguish between classes if the input features lack discriminating information. This was one of the failure modes we noticed early on.

In the crisis branch errors most likely come from noise in public reports. Tweets may use expressive language, may not describe the event completely, or may include images that don't clearly show the disaster condition. Some crisis classes are also harder to separate than others — affected persons, rescue needed, infrastructure failures, and general disaster scenes can look similar both visually and semantically. A model might correctly identify that a tweet contains disaster-related information but still struggle to classify the specific type of humanitarian crisis. For these reasons crisis branch

predictions should be evaluated carefully, they shouldn't be treated as ground truth. We learned this the hard way during our initial evaluations.

In the satellite branch errors are related to image quality, building density, and the extent of observable damage. Partial damage is harder to identify than completely destroyed buildings since cracked roofs or walls may not be distinguishable at the provided resolution. Shadows, vegetation, clouds, and poor image alignment can also mess with the satellite model. Still the satellite branch gives us useful information for disaster assessments even when perfect segmentation isn't achieved. In our fusion architecture the satellite model passes damage-aware embeddings to the fusion layer and this embedding can help with priority and severity estimation even if the pixel-level classifier makes mistakes on individual buildings.

Fusing the models together helps address a lot of these error cases. If the environmental branch is unsure between storm and flood, the crisis text might mention injuries or flooded streets, or satellite imagery might show buildings surrounded by water. If a tweet doesn't describe the event clearly the accompanying environmental metadata can offer context about recent weather conditions. If satellite imagery is unavailable due to timing constraints or the disaster occurs in a rural area with few buildings, the model can fall back on crisis and environmental data. It's not foolproof, we know that. But it does mean the model isn't stuck relying on one form of potentially noisy input. That's the whole point of the multimodal approach and why we went through all the trouble of building this.

For this reason we think case-level interpretation of predictions matters a lot. For each prediction the system makes it would be useful to know which modalities were used, which modalities had high confidence, and whether the modalities agreed with each other. A multimodal-supported high-priority prediction should be considered more reliable than a unimodal (crisis only) high-priority prediction based on a subjective social media post. If the model estimates low priority but satellite imagery shows large areas of severe building damage the case could be flagged for review by a humanitarian worker. That kind of cross-checking is what makes the system actually useful in practice.

We want to be clear that this platform is best used as a decision-support system rather than a standalone decision-maker. The model's job is to organize available

evidence, estimate likely severity, and direct human attention toward the most urgent cases. Humanitarian workers should use their training and situational knowledge to make final decisions, cross-referencing model predictions with field reports, on-the-ground knowledge, and institutional response guidelines. We can't stress this enough.

6.9 OPERATIONAL RELEVANCE AND ETHICAL CONSIDERATIONS

This platform is operationally relevant because emergency response often requires rapid interpretation of incomplete signals. At the start of a disaster responders may receive small patches of related incoming reports. Some citizens might be reporting emergencies on a platform. Some automated environmental sensors might be triggering alerts. Some satellite imagery might have been flagged by a remote sensing algorithm. A system that pulls these signals together into one assessment could cut down the reliance on slow manual intake processes, and that's really what we're trying to solve here. The manual process is just too slow when lives are at stake.

One concrete application is early prioritization. If a large number of related reports come in at once how should emergency teams decide where to focus? The priority prediction head can crank out automated prioritization scores based on the available evidence. Ideally the model would never fully replace human judgment. Instead responders could use it as triage assistance — automatic predictions help surface critical or resource-heavy cases to human operators who then evaluate the supporting evidence and make the call. We think this is probably the most immediately useful part of the whole system.

Evidence consolidation is another practical use case. One field report may say flooding has occurred, sensor metadata may show high rainfall amounts near the location, and satellite imagery arriving hours later may show damaged buildings. If these tools exist in silos those pieces of evidence may never be connected, which happens more often than you'd think. This platform pulls multimodal evidence into one unified view at the case level. What this means is it could improve situation awareness and prevent important evidence from being missed. That's a real problem in disaster response and one we heard about repeatedly when reading the literature.

Ethical considerations matter here too. Intelligence about disasters and emergencies can influence decisions about rescue efforts, resource distribution, and

citizen safety. If the model gets a prediction wrong the consequences can be real. Labeling something as low priority when it's actually urgent could delay care for people who need it. Labeling something as high priority when it's actually low-risk could pull resources away from cases that need them more. For these reasons model predictions should always come with a confidence measure and explanation summaries so that decision-makers understand the automation provides information, not decision-making authority. We thought about this a lot.

Privacy is another concern. User-submitted content on social media and public reporting tools might include personally identifiable information, location data, or images of victims. To my knowledge no such data was used from these sources in this project. But if the system were deployed in real emergency response situations privacy and security controls would need to be built in. Administrators should practice data minimization and access control. The platform shouldn't display personal information in dashboards or automated reports. If this project were extended for actual emergency use sensitive data handling policies would need to meet institutional and legal standards. We didn't have to deal with this directly but it would be the first thing to address in a real deployment.

Data bias is also worth thinking about. Social media datasets will always skew toward areas with internet access and miss vulnerable communities without connectivity. Satellite imagery might be purchased for some regions and not others. Sensor observations only exist where devices have been placed and maintenance gaps break their availability. Public crowdsourced datasets may carry historical bias in where users have operated. Because of this missing evidence from any data source should not be read as a negative result — the absence of a detected signal doesn't mean the absence of a disaster. When a piece of evidence is missing the model should reflect that uncertainty. This is something we kept coming back to throughout the project.

In the end intelligent emergency response systems should always keep humans in the loop. Models like this one can provide organized evidence and risk estimates but decision-making should be left to responders. The frontend dashboard was built with ease of review in mind. Operators should be able to quickly see what data went into a prediction, look at the explanation, and contact emergency services if the model's output contradicts on-the-ground knowledge. That's what keeps the tool useful without causing unintended consequences from misapplied AI. And that's really the bottom line.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 CONCLUSION

This project produced a working multimodal disaster assessment prototype. It takes in environmental data, crisis-media input, and satellite imagery through a single pipeline and shows the fused output on a web dashboard. That was the goal and we got there, though it took longer than we expected.

The repository now includes the branch models, fusion code, explanation outputs, the API layer, saved-case handling, and dashboard pages. In recorded results the full tri-modal setup outperforms the crisis-only setting, particularly on priority and severity. Those are the metrics that matter most for disaster response.

So the final outcome is more than a trained model. It's a runnable prototype that can be improved further in later work.

7.2 FUTURE WORK

The first next step is testing the environmental branch on live or near-real-time sensor streams rather than only archived datasets. That would show how the current weighting logic holds up under noisy values, missing readings, and timing gaps, stuff we couldn't test with the data we had.

A second extension worth pursuing is better satellite training. Higher-resolution runs, improved class balancing, and stronger hardware support should help with the minority damage classes. It could also raise fusion branch performance overall, which would be a nice bonus.

Over a longer horizon the model should be evaluated on naturally aligned tri-modal disaster data. That would remove the current reliance on synthetic pairing and give a more honest picture of how the system handles real events. That's ultimately what matters and it's the test we'd most want to see.

REFERENCES

Centre for Research on the Epidemiology of Disasters (CRED), "2023 Disasters in Numbers," CRED/UCLouvain, Brussels, Belgium, 2024.

T. Sakaki, M. Okazaki, and Y. Matsuo, "Earthquake shakes Twitter users: Real-time event detection by social sensors," in Proc. 19th Int. Conf. World Wide Web (WWW), 2010, pp. 851-860.

F. Alam, F. Ofli, and M. Imran, "CrisisMMD: Multimodal Twitter datasets from natural disasters," in Proc. 12th Int. AAAI Conf. Web Social Media (ICWSM), 2018, pp. 465-473.

M. Imran, C. Castillo, F. Diaz, and S. Vieweg, "Processing social media messages in mass emergency: A survey," ACM Computing Surveys, vol. 47, no. 4, pp. 1-38, 2015.

R. Gupta et al., "xBD: A dataset for assessing building damage from satellite imagery," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW), 2019.

Defense Innovation Unit, "xView2 Challenge: Assessing building damage," 2019. [Online]. Available: https://xview2.org

E. Weber and H. Kan, "Building disaster damage assessment in satellite imagery with multi-temporal fusion," in Proc. IEEE Int. Conf. Big Data, 2020.

A. Olteanu, C. Castillo, F. Diaz, and S. Vieweg, "CrisisLex: A lexicon for collecting and filtering microblogged communications in crises," in Proc. 8th Int. AAAI Conf. Weblogs and social media (ICWSM), 2014.

M. Abavisani et al., "Multimodal categorization of crisis events in social media," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2020.

R. M. Allen, "Real-time earthquake detection and hazard assessment by ElarmS across California," Geophysical Research Letters, vol. 36, no. 5, 2009.

E. Basha and D. Rus, "Design of early warning flood detection systems for developing countries," in Proc. Int. Conf. Information and Communication Technologies and Development (ICTD), 2008.

F. Ofli, F. Alam, and M. Imran, "Analysis of social media data using multimodal deep learning for disaster response," in Proc. 17th Int. Conf. Information Systems for Crisis Response and Management (ISCRAM), 2020.

T. G. J. Rudner et al., "Multi3Net: Segmenting flooded buildings via fusion of multiresolution, multisensor, and multitemporal satellite imagery," in Proc. AAAI Conf. Artificial Intelligence, 2019.

A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), 2017.

J. Lu, D. Batra, D. Parikh, and S. Lee, "ViLBERT: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2019.

H. Tan and M. Bansal, "LXMERT: Learning cross-modality encoder representations from transformers," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2019.

J. Li, D. Li, C. Xiong, and S. Hoi, "BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation," in Proc. Int. Conf. Machine Learning (ICML), 2022.

H. Chefer, S. Gur, and L. Wolf, "Transformer interpretability beyond attention visualization," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2021, pp. 782-791.

H. Mouzannar, Y. Rizk, and M. Awad, "Guided cross-attention for multimodal crisis classification with LLaVA-generated captions," Information Processing & Management, vol. 62, 2025.

V. Benson, A. Ecker, and R. Schmid, "Multi-scale damage assessment on xBD: Advancing building damage segmentation with Dice-Focal loss," in Proc. IEEE Int. Geoscience and Remote Sensing Symposium (IGARSS), 2025.

Dewminim Naadi, "Sri Lanka Flood Risk & Inundation Dataset," Kaggle, 2024. [Online]. Available: https://www.kaggle.com/datasets/dewminimnaadi/sri-lanka-flood-risk-and-inundation-dataset

The Devastator, "Tropical Cyclone Tracks Dataset," Kaggle, 2023. [Online]. Available:https://www.kaggle.com/datasets/thedevastator/tropical-cyclone-tracks-dataset.

A. Khaled, "California Weather and Fire Prediction Dataset," Kaggle, 2023. [Online]. Available: https://www.kaggle.com/datasets/aleenkhaled/california-weather-and-fire-prediction-dataset

CORGIS Datasets Project, "Earthquakes Dataset," Virginia Tech, 2023. [Online]. Available: https://corgis-edu.github.io/corgis/csv/earthquakes/

John Tukey, "Dataset of Earthquakes in Iran (1996-2022)," Kaggle, 2023. [Online]. Available: https://www.kaggle.com/datasets/johntukey/iran-earthquake

Utkarsh, "NOAA Atlantic Hurricane Dataset (1975-2024)," Kaggle, 2024. [Online]. Available: https://www.kaggle.com/datasets/utkarshx27/noaa-atlantic-hurricane-database

APPENDICES

APPENDIX A REPOSITORY SUMMARY

The project repository is organized around the environmental, crisis, satellite, and fusion subsystems. Evaluation outputs and earlier publication material are stored alongside the code so that experiments can be reproduced and the documentation can reference recorded metrics directly. We tried to keep things self-contained.

The main directories are `IOT/` for the environmental branch, `crisis/` for the social-media branch, `XBD/` for the satellite branch, `fusion/` for the tri-modal serving and integration layer, `outputs-paper/` for evaluation artifacts, and `paper/` for earlier publication drafts. Standard stuff.

APPENDIX B KEY EXECUTION COMMANDS

Representative training commands from the repository are `python IOT/train_iot.py` for the environmental metadata model and `python fusion/train_tri_fusion.py` for the tri-fusion layer. The crisis and satellite branches use notebook-based workflows instead, which was just easier for iterating on those parts.

The server can be started through the fusion subsystem and the static assets under `fusion/static/` give us the browser-side interface where users view results.

APPENDIX C MODEL CONFIGURATION SUMMARY

The system has four main model components. The environmental branch takes a 32-dimensional numerical vector and maps grouped sensor features into a learned embedding. The crisis branch takes an image-text pair and spits out a multimodal crisis embedding. The satellite branch takes post-disaster imagery and outputs damage-aware satellite features. The tri-fusion layer then pulls all three embeddings together and produces multi-task predictions.

In the environmental branch, weather, storm, seismic, and hydrological features are encoded separately before being combined, which lets the model learn behavior specific to each feature family. The crisis branch runs separate image and text encoders followed by cross-modal fusion. The satellite branch is built on a modified DeepLabV3+ architecture that supports damage representation learning. The fusion layer applies pairwise cross-attention and adaptive gating across modalities. Each of these took a fair bit of tuning to get right.

The system outputs a disaster type, severity score, priority level, population impact estimate, and resource-related estimate, all produced from the fused representation. It is a multi-task design because disaster response needs a few different kinds of information, not just a single class label. One label wouldn't cut it for what we're trying to do.

APPENDIX D SAMPLE INFERENCE WORKFLOW

A typical inference run starts when a user opens the dashboard and creates a new disaster case. They can upload a crisis image, type in a public report, fill in environmental values, and optionally attach satellite imagery. The frontend sends these inputs to the backend which checks what data is available and prepares each input for the right model branch.

The environmental values are packed into a fixed numerical vector. The crisis image is resized and normalized and the report text is tokenized for the language encoder. The satellite image is resized and fed to the satellite model. Each branch produces its own embedding and these get passed into the tri-fusion layer to compute a fused representation.

The prediction heads then kick out the final outputs. The dashboard displays the predicted disaster type, severity, priority, and related estimates. If explanation data is available it also shows which modalities had the most influence. The operator can then review the case and decide what to do with it.